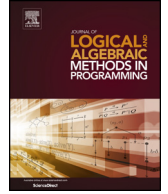




Contents lists available at ScienceDirect

Journal of Logical and Algebraic Methods in Programming

journal homepage: www.elsevier.com/locate/jlamp

A dynamic logic with branching modalities

Yuanrui Zhang^a, Zhiming Liu^{b,*}^a School of Mathematics and Statistics, Southwest University, China^b College of Computer and Information Science, Southwest University, China

ARTICLE INFO

Article history:

Received 3 February 2023

Received in revised form 28 June 2023

Accepted 27 September 2023

Available online 2 October 2023

Keywords:

Dynamic logic

Temporal property

Branching semantics

Program verification

ABSTRACT

We propose a dynamic logic DL_b called “dynamic logic with branching modalities”, which extends the temporal dynamic logic DLT with a “branching modality” for specifying safety properties of regular programs with tests (simply “regular programs”). Compared to the trace modality of DLT for *while* programs that do not abort, branching modality of DL_b does not exclude aborting traces introduced by regular programs, thus is able to capture a type of safety properties which are important for systems with failure behaviors. Moreover, it is congruent to the compositionality of regular programs so that the proof system naturally extended from that of DLT is proved to be complete for DL_b . In this paper, we build the theory of DL_b on both propositional and first-ordered levels, defining two logics: propositional DL_b (PDL_b) and first-ordered DL_b ($FODL_b$). PDL_b forms the theoretical basis of DL_b while $FODL_b$ is useful for practical verification. We propose the proof systems for PDL_b and $FODL_b$, and analyze their decidability, soundness and (relative) completeness in a formal way, through comparing their expressiveness and deduction capabilities with propositional dynamic logic (PDL) and first-order dynamic logic (FODL) respectively. We show that $FODL_b$ is actually an extension of DLT, and illustrate the motivations of using the branching modality through an example.

© 2023 Elsevier Inc. All rights reserved.

1. Introduction

Dynamic logic [1] is a mathematical logic for formally specifying and reasoning about computer programs. Unlike traditional modal logics like LTL and CTL [2] which describe general properties for all systems, dynamic logic describes properties for a specific system, expressed as a certain structure called *program model* in the logic. Usually, a property in dynamic logic has the form $\psi \rightarrow [\alpha]\phi$, where ψ and ϕ are formulas and α is a program model embedded in a *modality* $[\cdot]$. Formula $\psi \rightarrow [\alpha]\phi$ expresses that if ψ holds, then property ϕ holds after the termination of any execution of program α . The triple $\{\psi\}\alpha\{\phi\}$ in Hoare logic [3] has exactly the same meaning as $\psi \rightarrow [\alpha]\phi$. But compared to Hoare logic, dynamic logic is more expressive and is in a more flexible form since program is taken as a part of the logic itself. With programs as “first-class citizens”, dynamic logic provides a natural way to reason about programs. It now has been applied, or extended to be applied to specification and verification of many actual programs or computer system models, such as Java [4], process algebras [5,6], hybrid systems [7,8], probabilistic systems [9,10], epistemic models [11], game models [12,13] and so on.

One promising research topic is to marry dynamic logic and temporal logic [14]. The former can directly reason about programs, while the latter has the ability to describe temporal properties, which are one of crucial types of properties for

* Corresponding author.

E-mail addresses: zhangyrmath@swu.edu.cn (Y. Zhang), zhimingliu88@swu.edu.cn (Z. Liu).

computer systems, especially for real-time ones. An early attempt is process logic [15], it extends propositional dynamic logic [16] with temporal formulas. Later, [17] proposed a variation of process logic on the first-ordered level, called *dynamic logic with trace modalities* (DLT), for deterministic *while* programs. DLT extends a deterministic dynamic logic [18] with a trace modality $\llbracket \cdot \rrbracket$, which captures safety properties — an important type of temporal properties — of a program. A formula $\llbracket \alpha \rrbracket \phi$ in DLT expresses that property ϕ holds at *every intermediate state* of any execution trace of program α . DLT has a sound and complete proof system relative to arithmetic first-ordered logic (AFOL). It has also been implemented as a part of the theorem prover KeY [19], and has been successfully used for verifying Java programs with considerable sizes.

In this paper, we propose to extend the *while* programs of DLT to the program model *regular programs with tests*, or simply *regular programs*. Regular program or its variations have been proved to be useful formalisms in specifying abstract system behaviors and system failures, and providing a support for system refinements. And they have been adopted in variations of dynamic logic for specifying system behaviors, such as [7,13,20–22]. We propose a novel temporal dynamic logic called DL_b , whose first-ordered level is useful for specifying and verifying safety properties of regular-program models.

A difference in expressiveness between the *while* programs of DLT and regular programs is that regular programs can generate *aborting traces*: traces that are blocked by dissatisfaction of tests without terminations. When the trace modality $\llbracket \cdot \rrbracket$ of DLT is adapted to regular programs, there would be two main problems:

The first is that the semantics of $\llbracket \cdot \rrbracket$ would exclude these aborting traces, thus making a property $\llbracket \alpha \rrbracket \phi$ not a safety property, in the sense that not all intermediate states of α have to satisfy formula ϕ . Since a regular program can fail, sometimes we also need to check those intermediate states that only lead to abortions. This always happen when modeling the failure behaviors of a real-time system or a reactive system [23], when we want to know what happened before the system fails or what causes a failure of the system (see Example 1).

Secondly, the excluding-aborting-traces semantics of $\llbracket \cdot \rrbracket$ would not be congruent to the compositionality of regular programs. In other words, a safety property, say $\llbracket \alpha; \beta \rrbracket \phi$, holds for a program $\alpha; \beta$ does not mean that safety property $\llbracket \alpha \rrbracket \phi$ holds for program α . This is a more serious problem. As is fully explained in Sect. 6.3, the sequential compositional rule ($\llbracket \cdot \rrbracket$), which is extended naturally from DLT for regular programs, would become too strong, so that the whole proof system extended naturally from DLT is not complete related to AFOL.

For the above two reasons, in this paper, we propose a so-called *branching modality*, denoted by $\llbracket \cdot \rrbracket$, as the semantics of modality in DL_b to cover all aborting traces of regular programs. DL_b is actually the abbreviation of *dynamic logic with branching modalities*. As indicated by its name, branching modality considers every intermediate state in all *branches* starting from a state. For a regular program α , $\llbracket \alpha \rrbracket \phi$ captures a safety property in which every intermediate state, no matter whether leading to terminations or abortions, satisfies ϕ , which has exactly the same meaning as the trace modality $\llbracket \cdot \rrbracket$ in DLT. Branching modality $\llbracket \cdot \rrbracket$ is congruent to the compositionality of regular programs. It makes the direct extension of the proof system of DLT possible, in the sense that the compositional rules ($\llbracket \cdot \rrbracket$) and ($\llbracket \cup \rrbracket$) of DLT can be directly extended to the case for regular programs in PDL_b .

This paper mainly focuses on developing the theory of DL_b . We start from the propositional level of DL_b — propositional DL_b (PDL_b), which forms the theoretical basis of DL_b . The syntax of PDL_b is defined and a complete proof system for PDL_b is proposed. In terms of expressiveness, PDL_b stays the same as propositional dynamic logic (PDL) [16]. We prove that essentially, each dynamic formula $\llbracket \alpha \rrbracket \phi$ can be encoded as a PDL formula (Proposition 2). In the proof system of PDL_b , we further show that PDL_b formulas and PDL formulas are inter-provable (Proposition 4). This indicates a direct way of proving the completeness of PDL_b according to the completeness of PDL.

Based on the theory of PDL_b , we define the first-ordered level of DL_b — first-ordered DL_b ($FODL_b$) — and propose a relatively complete proof system for $FODL_b$. $FODL_b$ provides a framework for verifying safety properties of a regular program. A safety property described as a $FODL_b$ dynamic formula is verified by semi-automatically transformed into a set of AFOL formulas. These AFOL formulas can then be checked through a (semi-)automatic procedure on a SAT/SMT solver in the usual way. As $FODL_b$ can be seen as a more special case of PDL_b , its expressibility and its proof system's capability compared to FODL are similar to those of PDL_b compared to PDL. We analyze their relationship, based on which the relative completeness of $FODL_b$ is proved.

Our proposed $FODL_b$ actually can be seen as an extension of DLT. We show this by proving that $FODL_b$ subsumes DLT in terms of expressiveness (Proposition 7). Intuitively, this is due to the fact that the *while* programs in DLT do not allow free tests and thus do not generate aborting traces. Therefore, the traces semantics of a *while* program represents the same set of intermediate states as the corresponding branching semantics of the same program. On the other hand, since regular programs can generate aborting traces, we show through a counter example (Sect. 6.3) that the sequential compositional rule ($\llbracket \cdot \rrbracket$) of DLT would be too strong when adapted to regular programs.

The main contributions of this paper can be summarized as the following three points:

1. We propose a temporal dynamic logic DL_b with a new type of modalities, called branching modality, which is useful for specifying and reasoning about safety properties of regular program models. We build DL_b on both the propositional and first-ordered level, defining the logics PDL_b and $FODL_b$, and propose suitable proof systems for them.
2. We analyze the decidability, expressiveness, soundness and (relative) completeness of both PDL_b and $FODL_b$ in a formal way, and give their proofs.

3. We make a comparison between $FODL_b$ and DLT, and prove that $FODL_b$ is in fact an extension of DLT in terms of expressiveness. We analyze why an attempt to naturally extending DLT from its trace semantics would fail, which motivates our propose of the branching modality.

The rest of the paper is organized as follows. Sect. 2 introduces related work that is mostly related to ours. Sect. 3 gives a brief introduction to several previous logics mentioned above and sequent calculus, which are necessary for understanding this paper. Sect. 4 introduces the theory of PDL_b , while in Sect. 5 we propose the theory of $FODL_b$. A comparison between $FODL_b$ and DLT is made in Sect. 6. Sect. 7 concludes the paper and discusses some future work. The proofs of all propositions, lemmas and theorems are given in Appendix A, Appendix B and Appendix C.

2. Related work

The proposed DL_b can be seen as an extension from DLT [17] for regular programs with tests. Except for DLT and other variations of dynamic logics as already mentioned in Sect. 1, we list some other works that are closely related to ours.

Process logic was firstly proposed in [15] as a fusion of dynamic logic and temporal logic. Based on some earlier works like [24], it extends propositional dynamic logic with temporal formulas and has a trace semantics for both of its programs and logical formulas. For example, a trace tr satisfies a formula $[\alpha]\Box\phi$ (in which $\Box$ is the standard “always” operator in LTL [2]) in process logic means that for any execution trace tr' of α , the concatenation $tr \cdot tr'$ between two traces (an operator also defined in Sect. 6.1) satisfies $\Box\phi$. This is different from DL_b , as the semantics of DL_b formulas is based on states (see Definition 2). Process logic has a sound and complete proof system, but concurrently, only the propositional level of the logic has been fully studied.

In [25], the authors proposed a dynamic trace logic that enriches DLT with linear temporal formulas to express a larger set of temporal properties, not just safety ones. In dynamic trace logic, formula $[\alpha]\Box\phi$ has the same meaning as formula $\llbracket\alpha\rrbracket\Box\phi$ in DLT, but formula like $[\alpha]\phi\mathcal{U}\psi$ (in which $\mathcal{U}$ is the standard “until” operator in LTL [2]) can express a liveness property. Dynamic trace logic has a sound and relatively complete proof system, which extends the proof system of DLT with rules for additional temporal formulas like $\phi\mathcal{U}\psi$. Like DLT, the program model of dynamic trace logic is the *while* program model. So its semantics rejects aborting traces that are generated in regular programs with free tests.

Dynamic temporal differential logic (dTL) is a logic for verification of hybrid systems [26]. Like process logic, it extends differential dynamic logic (dL) [7] with temporal formulas, such as $\Box\phi$ and $\Diamond\phi$ (where $\Diamond$ is the standard “eventually” operator in LTL [2]), to capture temporal properties of hybrid systems. dTL can describe both safety and liveness properties. Based on dTL, [27] proposed a logic called dTL^2 , which can further derive some temporal properties that cannot be derived in dTL. Still, following DLT, the semantics of the program models of dTL and dTL^2 is based on traces that exclude aborting traces. Hybrid program in dTL and dTL^2 is a more specific model than regular program with tests in DL_b . Though DL_b takes a different semantics based on branching states rather than traces, the rules ($[\cdot]; [\cdot]$) and ($[\cdot] * [\cdot]$) in PDL_b are directly inherited from dTL.

3. Preliminaries

In this section, we give a brief introduction to the syntax and semantics of propositional dynamic logic (PDL). And we give an informal introduction to first-ordered dynamic logic (FODL) and dynamic logic with trace modalities (DLT), which are closely related to our proposed logic in this paper.

3.1. PDL & FODL

The syntax of PDL is defined as follows in BNF form:

$$\begin{aligned}\alpha &::= a \mid \psi? \mid \alpha; \alpha \mid \alpha \cup \alpha \mid \alpha^*, \\ \phi &::= p \mid tt \mid \neg\phi \mid \phi \wedge \phi \mid [\alpha]\phi.\end{aligned}$$

α is a program model, defined as the set of regular programs with tests. a is an atomic program. $\psi?$ is a test. A program proceeds iff proposition ψ is true. $;$ is called *sequence operator*, $\alpha; \beta$ means that after program α terminates, program β proceeds. $\cup$ is *non-deterministic choice operator*, $\alpha \cup \beta$ means that either program α or β proceeds. $*$ is *star operator*, program α^* executes α for an arbitrary finite number of times.

p is an atomic proposition. tt is the Boolean true. We also use ff to represent the Boolean false, which is $\neg tt$. Formula $[\alpha]\phi$ is called a *dynamic formula*, it captures a state property about the program α , saying that formula ϕ is satisfied after each termination of program α .

The semantics of PDL is given by a Kripke frame (S, m) (cf. [28]), where S is a set of states (or called *worlds*), m is a mapping that maps each atomic program to a set of state pairs, and maps each atomic proposition to a set of states. The mapping m for the other programs and formulas are inductively defined as follows:

- $m(\psi?) =_{df} \{(s, s) \mid s \in m(\psi)\};$

- $m(\alpha; \beta) =_{df} \{(s, t) \mid \text{there exists a } u \text{ such that } (s, u) \in m(\alpha) \text{ and } (u, t) \in m(\beta)\};$
- $m(\alpha \cup \beta) =_{df} m(\alpha) \cup m(\beta);$
- $m(\alpha^*) =_{df} \bigcup_{n \geq 0} m(\alpha^n)$, where $\alpha^0 =_{df} tt?$, $\alpha^k =_{df} \underbrace{\alpha; \dots; \alpha}_k$ for $k > 0$;
- $m(tt) =_{df} S;$
- $m(\neg\phi) =_{df} S - m(\phi);$
- $m(\phi \wedge \psi) =_{df} m(\phi) \cap m(\psi);$
- $m([\alpha]\phi) =_{df} \{s \mid \text{for all } s', \text{ if } (s, s') \in m(\alpha), \text{ then } s' \in m(\phi)\}.$

The symbols $\cup$, $\cap$ and $-$ appeared on the right hand of the equations represent the usual union, intersection and complement operators between sets.

FODL [29] is a dynamic logic explained in a first-ordered signature, over which first-ordered quantification is allowed. Its syntax is as follows:

$$\begin{aligned} \alpha &::= x := e \mid \psi? \mid \alpha; \alpha \mid \alpha \cup \alpha \mid \alpha^*, \\ \phi &::= p \mid tt \mid \neg\phi \mid \phi \wedge \phi \mid \forall x. \phi \mid [\alpha]\phi. \end{aligned}$$

$x := e$ is an *assignment*, meaning that assigning the value of expression e to variable x . In this paper, we restrict a test proposition ψ to a quantifier-free FODL formula in which no modality $[\cdot]$ occurs. The form of an atomic formula p depends on the domain we discuss. For example, in an arithmetical domain, formulas like $x > 2$, $x < y$ and $y - x < 0$ are all atomic formulas.

The semantics of FODL is defined as a Kripke frame (S, m) under an interpretation over the FODL signature. S is determined by the discussed domain. In Sect. 5.1 later, the semantics of FODL is shown as a part of the semantics of our proposed logic $FODL_b$.

3.2. Dynamic logic with trace modalities (DLT)

DLT [17] is a first-ordered dynamic logic which extends deterministic dynamic logics with a trace modality $\llbracket \cdot \rrbracket$, its syntax is as follows.

$$\begin{aligned} \alpha &::= x := e \mid \alpha; \alpha \mid \text{if } \psi \text{ then } \alpha \text{ else } \alpha \mid \text{while } \psi \text{ do } \alpha, \\ \phi &::= tt \mid a \mid \neg\phi \mid \phi \wedge \phi \mid \forall x. \phi \mid [\alpha]\phi \mid \llbracket \alpha \rrbracket \phi. \end{aligned}$$

The program model of DLT is a deterministic one, we called *while programs*. Compared to regular programs in FODL, a *while* program does not include test statements, and has the statements *if...then...else...* and *while...do...* in place of the non-deterministic choice and star loop of regular programs respectively. The statements *if...then...else...* and *while...do...* have their usual meanings in programming languages. Dynamic formula $\llbracket \alpha \rrbracket \phi$ captures a safety property of a *while* program. Intuitively, $\llbracket \alpha \rrbracket \phi$ means that for any execution trace of α , all of its states satisfy property ϕ .

The semantics of DLT is based a trace semantics of DLT programs. We formally state it in Sect. 6.1.

3.3. Gentzen's sequent

In this paper, as is seen in the following sections, we adopt Gentzen's sequent to describe the inference rules of the proposed logics. We choose this logical form instead of Hilbert-style calculus, as our logic is intended for actual usage in practice. So sequent calculus is more suitable to our needs.

A sequent [30] is of the form $\Gamma \Rightarrow \Delta$, where both Γ and Δ are finite multi-sets of formulas, called *contexts*. A sequent can conveniently describe both top-down and bottom-up natural deductions as bottom-up deductions on the left and right sides of the arrow ' $\Rightarrow$ ' respectively. Intuitively, a sequent $\Gamma \Rightarrow \Delta$ represents a formula

$$\bigwedge_{\phi \in \Gamma} \phi \rightarrow \bigvee_{\psi \in \Delta} \psi,$$

which means that if all formulas in Γ hold, then one of formulas in Δ holds. We use ν to represent a sequent.

When $\Gamma = \emptyset$, we denote the sequent as $\cdot \Rightarrow \Delta$, meaning the formula $tt \rightarrow \bigvee_{\psi \in \Delta} \psi$. When $\Delta = \emptyset$, we denote the sequent as $\Gamma \Rightarrow \cdot$, meaning the formula $\bigwedge_{\phi \in \Gamma} \phi \rightarrow ff$.

Sometimes, we say a sequent ν is a "X sequent" if all formulas in ν are X formulas. For example, a PDL_b sequent, or a PDL sequent.

An inference rule in a sequent calculus is of the form

$$\frac{\Gamma_1 \Rightarrow \Delta_1 \quad \dots \quad \Gamma_n \Rightarrow \Delta_n}{\Gamma \Rightarrow \Delta},$$

where $\Gamma_1 \Rightarrow \Delta_1, \dots, \Gamma_n \Rightarrow \Delta_n$ are called *premises*, $\Gamma \Rightarrow \Delta$ is called a *conclusion*. The rule is *sound* if it satisfies that the validity of $\Gamma_1 \Rightarrow \Delta_1, \dots, \Gamma_n \Rightarrow \Delta_n$ implies the validity of $\Gamma \Rightarrow \Delta$.

Often, we use $\frac{\phi}{\psi}$ to represent a rule $\overline{\phi \Rightarrow \psi}$, which means that formula $\phi \rightarrow \psi$ is valid. The reason to write it in this form is that rule $\frac{\phi}{\psi}$ actually implies that both derivations $\frac{\Gamma \Rightarrow \phi, \Delta}{\Gamma \Rightarrow \psi, \Delta} (\downarrow)$ and $\frac{\Gamma, \phi \Rightarrow \Delta}{\Gamma, \psi \Rightarrow \Delta} (\uparrow)$ are rules, where Γ and Δ are arbitrary contexts. This fact is stated as Proposition 9 of Appendix A. If both $\frac{\phi}{\psi}$ and $\frac{\psi}{\phi}$ are rules, then we simply integrate them as a single rule: $\frac{\phi}{\psi}$, meaning that formula $\phi \leftrightarrow \psi$ is valid. We also call $\frac{\phi}{\psi}$ a *bidirectional* rule.

4. DL_b – a dynamic logic with branching modalities

We propose DL_b , a dynamic logic with branching modalities. In this section, we first introduce propositional DL_b (PDL_b) – DL_b on the propositional level. We define its syntax and semantics, and propose a sound and complete proof system for it. Then we compare its expressiveness with PDL, and analyze its decidability and completeness. Based on PDL_b , the first-ordered DL_b is discussed later in Sect. 5.

4.1. Syntax and semantics of PDL_b

PDL_b is extended from PDL with a modality $[[\cdot]]$, we call *branching modality*, for describing safety properties of a program. The syntax of PDL_b is given as follows.

Definition 1 (PDL_b formulas). The syntax of PDL_b formulas is defined as follows:

$$\begin{aligned} \alpha &::= a \mid \psi? \mid \alpha; \alpha \mid \alpha \cup \alpha \mid \alpha^*, \\ \phi &::= p \mid tt \mid \neg\phi \mid \phi \wedge \phi \mid [\alpha]\phi \mid [[\alpha]]\phi. \end{aligned}$$

Program α is just a regular program in PDL, in which ψ is a PDL_b formula in which no modalities $[\cdot]$ and $[[\cdot]]$ appear. The introduction of a test $\psi?$ in regular programs can abort a program without terminating, when the formula ψ cannot be satisfied by the current state. For example, formula $a; ff?$. Note that α^* can execute for “0 times”. In this case, α^0 does nothing and its effect is equivalent to $tt?$, as is seen in Definition 2 below.

Intuitively, dynamic formula $[[\alpha]]\phi$ means that any state *reachable from* α satisfies formula ϕ . A program reaches a state does not mean it can actually terminate on that state. This is the essential difference from other dynamic logics that are based on a trace semantics. Similarly in PDL, we introduce the modalities $\langle\alpha\rangle$ and $\langle[\alpha]\rangle$ as the duals of $[\alpha]$ and $[[\alpha]]$ respectively. Thus $\langle\alpha\rangle\phi =_{df} \neg[\alpha]\neg\phi$ and $\langle[\alpha]\rangle\phi =_{df} \neg[[\alpha]]\neg\phi$. Intuitively, $\langle\alpha\rangle\phi$ means that after some execution of α , ϕ holds, while $\langle[\alpha]\rangle\phi$ means that there exists some state reachable from α that satisfies ϕ .

In order to define the semantics of PDL_b , we extend the Kripke frame (S, m) for PDL with a mapping Re capturing the set of *reachable states* by a program. A Kripke frame for PDL_b is thus a triple (S, m, Re) .

To express the semantics of sequence programs in a more convenient way, we introduce the concatenation operator $\circ$. Given two state pairs (s, u) and (v, t) , the concatenation $\circ$ between them is a partial function defined as:

$$(s, u) \circ (v, t) =_{df} (s, t) \text{ if } u = v.$$

The concatenation can be lifted to an operator between two sets of state pairs. Given two sets of state pairs T_1 and T_2 , the concatenation between T_1 and T_2 , denoted by $T_1 \circ T_2$, is defined as

$$T_1 \circ T_2 =_{df} \{(s, u) \circ (v, t) \mid (s, u) \in T_1, (v, t) \in T_2\}.$$

The semantics of PDL_b is given below by simultaneous induction on both m and Re .

Definition 2 (Semantics of PDL_b). Given a Kripke frame (S, m, Re) , where m maps each atomic formula p to a subset of states of S , and maps each atomic program a to a set of state pairs over S , then $m(\gamma)$ and $Re(\gamma)$ for an arbitrary program γ of other forms are inductively defined as follows:

- $m(\psi?) =_{df} \{(s, s) \mid s \in m(\psi)\};$
- $m(\alpha; \beta) =_{df} m(\alpha) \circ m(\beta);$
- $m(\alpha \cup \beta) =_{df} m(\alpha) \cup m(\beta);$
- $m(\alpha^*) =_{df} \bigcup_{n \geq 0} m(\alpha^n)$, where $\alpha^0 =_{df} tt?$, $\alpha^k =_{df} \underbrace{\alpha; \dots; \alpha}_k$ for $k > 0$.

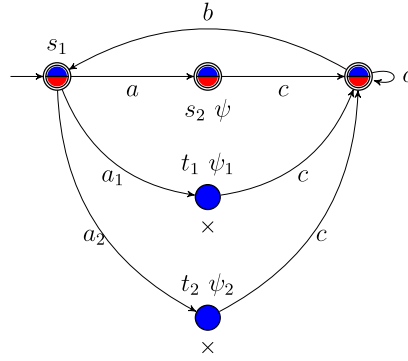


Fig. 1. A Kripke frame of program α with initial state s_1 . (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

- $Re(b) =_{df} m(tt?) \cup m(b)$, b is an atomic program a or a test ψ ;
- $Re(\alpha; \beta) =_{df} Re(\alpha) \cup m(\alpha) \circ Re(\beta)$;
- $Re(\alpha \cup \beta) =_{df} Re(\alpha) \cup Re(\beta)$;
- $Re(\alpha^*) =_{df} \bigcup_{n \geq 0} Re(\alpha^n)$.

$m(\varphi)$ for an arbitrary formula φ of other forms is defined as follows:

1. $m(tt) =_{df} S$;
2. $m(\neg\phi) =_{df} S - m(\phi)$;
3. $m(\phi \wedge \psi) =_{df} m(\phi) \cap m(\psi)$;
4. $m([\alpha]\phi) =_{df} \{s \mid \text{for all } s', \text{ if } (s, s') \in m(\alpha), \text{ then } s' \in m(\phi)\}$;
5. $m([\alpha]\phi) =_{df} \{s \mid \text{for all } s', \text{ if } (s, s') \in Re(\alpha), \text{ then } s' \in m(\phi)\}$.

A state pair $(s, s') \in m(\alpha)$ iff starting from s , program α terminates at state s' . Re maps each program to a set of *reachable states* from that program. A pair $(s, s') \in Re(\alpha)$ iff starting from s , program α can reach state s' , but not necessarily terminate at s' . $m(tt) \subseteq Re(b)$ because naturally we think an initial state s is reachable from itself, i.e., $(s, s) \in Re(b)$ for any $s \in S$. The critical part of Definition 2 is $Re(\alpha; \beta)$, where we can see the difference from $m(\alpha; \beta)$, that $Re(\alpha; \beta)$ captures all reachable states from α , and all reachable states after α successfully terminate.

The semantics of PDL_b formulas given in Definition 2 is defined as the same as PDL except for $[\alpha]\phi$, which expresses a safety property that each intermediate state reachable from α satisfies ϕ . $[\alpha]$ is different from the trace modality $\llbracket \alpha \rrbracket \phi$ in DLT, which is introduced later in Sect. 6.1, in that $[\alpha]$ also iterates those states that do not terminate. This is useful in practice for verifying a system that can possibly fail without termination. For example, as the following Example 1 shows, when we want to distinguish different cases of failures of a system.

Example 1. Consider a Kripke frame shown in Fig. 1 and a regular program $\alpha =_{df} \beta; \gamma$, where

$$\beta =_{df} a \cup a_1 \cup (\phi?; a_2),$$

$$\gamma =_{df} \psi?; c; c^*; b.$$

α is a non-deterministic program and not all of its executions terminate. $t_1 \notin m(\psi)$, $t_2 \notin m(\psi)$ and $s_1 \notin m(\phi)$. The program can fail at state t_1 because $t_1 \notin m(\psi)$, but cannot fail at state t_2 since $s_1 \notin m(\phi)$. Formulas ϕ , ψ , ψ_1 and ψ_2 satisfy that $m(\phi) = \emptyset$, $m(\psi) = \{s_2\}$, $m(\psi_1) = \{t_1\}$ and $m(\psi_2) = \{t_2\}$ respectively.

A system with failure behaviors is prevalent in real-time and reactive systems. For example, in program β , actions a , a_1 and a_2 can represent entering different engine modes s_2, t_1 and t_2 of an aircraft respectively, while the condition $\psi?$ in γ can represent a particular altitude, where only engine mode s_2 can work normally. Program α thus models the behaviors (which possibly fail) of the aircraft (β) under the specific condition of altitude (γ).

Consider a DL_b formula $[\alpha]([\alpha_2]\psi_2)$, which specifies that whether program α can, from any state (but here, the only possibility is from state s_1), execute action a_2 and fail at state t_2 , rather than t_1 . Checking this property is useful and important for the whole system α , as it tells how the engine of this aircraft fails under the specific altitude. Whether this property holds or not determines, for example, whether the engine can work normally at a broader condition of altitude: $\gamma' =_{df} (\psi \vee \psi_1)?; c; c^*; b$, which forms the regular program $\beta; \gamma'$.

From Example 1, we see that another advantage of branching modality $[\cdot]$ is that it is congruent to the compositionality of regular programs: any safety property of α , say $[\alpha]\varphi$, is also a safety property of β ($[\beta]\varphi$) and γ ($[\gamma]\varphi$). Because

Table 1
Rules of PDL_b .

$\frac{[\alpha](\psi \rightarrow \phi)}{[\alpha]\phi \rightarrow [\alpha]\psi} \quad ([\rightarrow])$	$\frac{[\alpha](\phi \wedge \psi)}{[\alpha]\phi \wedge [\alpha]\psi} \quad ([\wedge])$	$\frac{\psi \rightarrow \phi}{[\psi?]\phi} \quad (\psi?)$	$\frac{[\alpha][\beta]\phi}{[\alpha; \beta]\phi} \quad ([;])$	$\frac{[\alpha]\phi \wedge [\beta]\phi}{[\alpha \cup \beta]\phi} \quad ([\cup])$	$\frac{\phi \wedge [\alpha][\alpha^*]\phi}{[\alpha^*]\phi} \quad ([*])$
$\frac{\phi \Rightarrow [\alpha]\phi}{\Gamma, \phi \Rightarrow [\alpha^*]\phi, \Delta} \quad (ind)$	$\frac{\phi \Rightarrow \psi}{\Gamma, [\alpha]\phi \Rightarrow [\alpha]\psi, \Delta} \quad ([gen])$				
(a) rules from PDL					
$\frac{\phi \wedge [b]\phi}{[[b]]\phi} \quad 1 \quad ([a])$	$\frac{[[\alpha]]\phi \wedge [\alpha][[\beta]]\phi}{[[\alpha; \beta]]\phi} \quad ([;])$	$\frac{[[\alpha]]\phi \wedge [[\beta]]\phi}{[[\alpha \cup \beta]]\phi} \quad ([\cup])$	$\frac{[\alpha^*][[\alpha]]\phi}{[[\alpha^*]]\phi} \quad ([[*]])$		
(b) rules special in PDL_b					

¹ $b \in \{a, \psi?\}$.

Table 2
Rules in classical propositional logic.

$\frac{}{\Gamma, \phi \Rightarrow \phi, \Delta} \quad (ax)$	$\frac{\Gamma \Rightarrow \phi, \Delta \quad \Gamma, \phi \Rightarrow \Delta}{\Gamma \Rightarrow \Delta} \quad (cut)$	$\frac{\Gamma \Rightarrow \Delta}{\Gamma, \phi \Rightarrow \Delta} \quad (w)$	$\frac{\Gamma, \phi, \phi \Rightarrow \Delta}{\Gamma, \phi \Rightarrow \Delta} \quad (c)$
$\frac{\Gamma, \neg\phi \Rightarrow \Delta}{\Gamma \Rightarrow \phi, \Delta} \quad (\neg r)$	$\frac{\Gamma \Rightarrow \neg\phi, \Delta}{\Gamma, \phi \Rightarrow \Delta} \quad (\neg l)$	$\frac{\Gamma \Rightarrow \phi, \Delta \quad \Gamma \Rightarrow \psi, \Delta}{\Gamma \Rightarrow \phi \wedge \psi, \Delta} \quad (\wedge r)$	$\frac{\Gamma, \phi, \psi \Rightarrow \Delta}{\Gamma, \phi \wedge \psi \Rightarrow \Delta} \quad (\wedge l)$

branching modality $[[\cdot]]$ visits every intermediate states (colored by blue in Fig. 2) of the system. While the trace modality $[[\cdot]]$ of DLT, if adapted to regular programs (see the definition of $m([[a]])\phi$ in Sect. 6.3), would only iterate those states (colored by red) that lead to terminations. The loss of this congruence property on regular programs is the exact reason why the sequential compositional rule $([;])$ of DLT would become too strong if adapted to regular programs (see Sect. 6.3 for more detail).

There exists a relation between $m(\alpha)$ and $Re(\alpha)$, stated as the following proposition. Intuitively, it states that any terminated state after α can be reached from α .

Proposition 1. *Given a Kripke frame (S, m, Re) , for any regular program α , $m(\alpha) \subseteq Re(\alpha)$.*

Proved in Appendix C. In Proposition 1, clearly it might be $m(\alpha) \subset Re(\alpha)$, when a regular program can be aborted by a test statement.

We introduce the concept of satisfaction relation in PDL_b as follows.

Definition 3 (Satisfaction relation). Given a Kripke frame $K =_{df} (S, m, Re)$, the satisfaction relation $K, s \models \phi$ between a state $s \in S$ and a formula ϕ under K , is defined s.t.

$$K, s \models \phi \text{ iff } s \in m(\phi).$$

If $K, s \models \phi$ for any state s , write $K \models \phi$. If $K \models \phi$ for any K , we say ϕ is *valid* or is a tautology, and simply write $\models \phi$.

We call a formula ϕ is *satisfiable* if $K, s \models \phi$ for some K and s . Often, we simply write $s \models \phi$ instead of $K, s \models \phi$ if K is clear or not important in the context.

4.2. A proof system of PDL_b

We propose a proof system of PDL_b in the style of sequent calculus, which consists of two parts: rules for regular programs as shown in Table 1, and rules for classical propositional logic as shown in Table 2.

From Table 1, we can see that some rules are bidirectional while some are not.

The rules of type (a) are inherited from PDL (cf. [28]). We restate them in a form of derivation between sequents. Their meanings are not hard to be inferred from their semantics. For example, rule $([;])$ says that under any contexts Γ and Δ (which are not displayed in the rule itself), all terminating states of $\alpha; \beta$ satisfy ϕ iff all terminating states of α satisfy $[\beta]\phi$. Rule (ind) is the mathematical induction rule for star program α^* . Intuitively, it means that if we can prove that $\phi \rightarrow [\alpha]\phi$ holds (for all contexts), then we can prove that $\phi \rightarrow [\alpha^*]\phi$ holds at some given contexts Γ and Δ . Rule $([gen])$ introduces the dynamic parts $[\cdot]$ of a formula.

The rules of type (b) are special in PDL_b for decomposing regular programs in modality $[[\cdot]]$. Rule $([[a]])$ says that all reachable states of a program b satisfy ϕ , iff the states before and after the execution of b satisfy ϕ . Rule $([;])$ intuitively means that under any contexts Γ and Δ (not displayed in the rule), proving that all reachable states of $\alpha; \beta$ satisfy ϕ , is equivalent to proving that all reachable states of α satisfy ϕ , and after terminations of α , all reachable states of β satisfy ϕ . Rule $([\cup])$ is similar to $([\cup])$, which means that to prove all reachable states of $\alpha \cup \beta$ satisfy ϕ , is equivalent to prove

that all reachable states of α satisfy ϕ and all reachable states of β satisfy ϕ . Rule ($[| * |]$) says that all reachable states of α^* satisfy ϕ , iff after any times (including 0 time) of executions of α , all reachable states of α satisfy ϕ .

Table 2 shows the rules inherited from classical propositional logic. We omit the discussion about them. Note that we only list the primitive rules. The rules for the connectives $\vee$ and $\rightarrow$ are listed in Table 4 of Sect. 5.2.

We define deduction relation in PDL_b . Since the derivations in PDL_b are carried over sequents, the deduction relation is between sequents, rather than formulas.

Definition 4 (Deduction relation). A sequent ν can be proved in PDL_b , denoted by $\vdash \nu$, if it can be derived by applying rules in Table 1 and 2.

In a deduction, sometimes we simply use formula ϕ to represent sequent $(\cdot \Rightarrow \phi)$. For example, write $\vdash \phi$ instead of $\vdash (\cdot \Rightarrow \phi)$.

Definition 4 defines nothing more than a traditional meaning of “deduction relation” in logic theories, since sequent is just a special form of assertion and itself can be seen as a logical formula.

To conclude this subsection, we declare that the proof system of PDL_b is sound, stated in the following theorem.

Theorem 1 (Soundness of PDL_b). Given a PDL_b formula ϕ , if $\vdash \phi$, then $\models \phi$.

Since the proof system of PDL is sound (cf. [28]), we only need to discuss the soundness of the rules of type (b) in Table 1. We give the proof according to the semantics of PDL_b formulas in Definition 2. The details are shown in Appendix A.

4.3. Decidability and completeness of PDL_b

In this section, we prove that our proposed PDL_b is decidable and complete. We show this by demonstrating that in fact, PDL_b has the same expressiveness as PDL.

Although PDL_b extends PDL with the ability to express temporal properties, as indicated by the rules of type (b) in Table 1, intuitively each dynamic formula $[|\alpha|]\psi$ can be “decomposed” into formulas of the form $[|\beta|]\psi'$ or $[\beta'] [|\beta|]\psi'$ where β has a simpler syntactic structure than α . We have the following proposition.

Proposition 2 (Expressibility of PDL_b in PDL). Any PDL_b formula can be expressed by a PDL formula, i.e., for any PDL_b formula ϕ , there exists a PDL formula ψ such that

$$\models \phi \leftrightarrow \psi.$$

The proof of this proposition is proceeded by induction on the syntactic structure of ϕ , see Appendix C for more details.

By the decidability of PDL (cf. [28]), we know that checking whether a PDL_b formula ϕ is satisfiable or not is decidable, for there is a PDL formula ψ such that $\models \phi \leftrightarrow \psi$ and checking the satisfiability of ψ is decidable. Therefore, by Proposition 2 we automatically have the following theorem.

Theorem 2 (Decidability of PDL_b). It is decidable to check that for a PDL_b formula ϕ , whether there are some K and s such that $K, s \models \phi$.

Proof. By Proposition 2, there is a PDL formula ψ such that $\models \phi \leftrightarrow \psi$. By the decidability of PDL, it is decidable to choose some $K = (S, m)$ and $s \in S$ such that $K, s \models \psi$. By Definition 2, a Re can be defined according to K . So $K' = (S, m, Re)$ is the Kripke frame we choose and certainly $K', s \models \phi$. $\square$

The proof system for PDL, consisting of the rules in Table 1 and 2 but excluding the rules of type (b), is proved to be complete (cf. [28]). We have the following statement.

Proposition 3 (Completeness of PDL (cf. [28])). For any PDL formula ψ , if $\models \psi$, then $\vdash \psi$.

Based on Proposition 3, as is seen below, we prove the completeness of the proof system of PDL_b by showing that we can derive the equivalence between a PDL_b formula and a PDL formula that is semantically equivalent to it, as stated in the next proposition.

Proposition 4. For any PDL_b formula ϕ , there is a PDL formula ψ such that $\vdash \phi \leftrightarrow \psi$.

Proposition 3 is actually a stronger proposition than Proposition 2 by the soundness of the proof system of PDL_b by Theorem 1. Note that $\vdash \phi \leftrightarrow \psi$ is stronger than $\models \phi \leftrightarrow \psi$, as from $\vdash \phi \leftrightarrow \psi$, $\models \phi \leftrightarrow \psi$ can be derived by applying rule $(\rightarrow r)$

of Table 4 (which is a derived rule from the rules in Table 2). While $\vdash \phi \Leftrightarrow \psi$ cannot be derived from $\vdash \phi \leftrightarrow \psi$ in our proof system.

In Appendix B, we prove Proposition 4 based on induction on the syntactic structure of PDL_b formulas and regular programs.

Theorem 3 is a direct result from Proposition 4 and Proposition 3.

Theorem 3 (Completeness of PDL_b). *Given a PDL_b formula ϕ , if $\models \phi$, then $\vdash \phi$.*

Proof. Given a PDL_b formula ϕ , by Proposition 4, there is a PDL formula ψ such that $\vdash \phi \Leftrightarrow \psi$. By the derived rule ($\rightarrow r$) of Table 4, we can derive $\vdash \phi \leftrightarrow \psi$. By the soundness of the proof system of PDL_b in Theorem 1, we have $\models \phi \leftrightarrow \psi$. If $\models \phi$, then $\models \psi$. By Proposition 3, $\vdash \psi$. By $\vdash \psi \Rightarrow \phi$ and $\vdash \psi$, we then can prove $\vdash \phi$ by applying rule (*cut*):

$$\frac{\cdot \Rightarrow \psi \quad \psi \Rightarrow \phi}{\cdot \Rightarrow \phi} \text{ (cut)} \quad \square$$

5. First-ordered DL_b

In this section, we propose first-ordered DL_b ($FODL_b$) — DL_b on the first-ordered level. $FODL_b$ is a predicate logic as a “precise version” of PDL_b in the arithmetical domain. First-ordered dynamic logics are an important type of logics as they are mostly concerned in the verification of computer programs. In the following we mainly focus on $FODL_b$, based on PDL_b , defining its syntax and semantics, and proposing a sound proof system for $FODL_b$. This proof system, though not complete due to Gödel’s incompleteness theorem, is proved to be complete *relative to* AFOL. Further, as stated in Sect. 1, we make a full comparison with DLT and show that $FODL_b$ actually subsumes DLT.

As seen in Sect. 4, we propose the branching modality $[\cdot \cdot]$ to capture safety properties of regular programs. This is different from DLT’s trace modality $[\cdot]$. In this section, we see that why the trace modality does not work for regular programs on the first-ordered level. And this is exactly the reason why the branching modality, as one of our main contributions, plays a crucial role in our logic.

5.1. Syntax and semantics of $FODL_b$

A general first-order dynamic logic [29] is defined under a signature, which consists of an arbitrary set of function symbols, predicate (or called “relation”) symbols and a countable set of variables of some domain. In this paper, we confine ourselves to signatures that have the same expressiveness as Peano’s arithmetic theory. And for simplicity, we only consider the integer domain $\mathbb{Z}$. Logics defined under these signatures are expressive enough to describe any data structures or computations in the field of computer programming.

The syntax of $FODL_b$ is as follows.

Definition 5 ($FODL_b$ formulas). The syntax of $FODL_b$ formulas is defined as follows:

$$\begin{aligned} e &::= x \mid c \mid f(e_1, \dots, e_n), \\ \alpha &::= x := e \mid \psi? \mid \alpha; \alpha \mid \alpha \cup \alpha \mid \alpha^*, \\ \phi &::= \theta(e_1, \dots, e_n) \mid tt \mid \neg \phi \mid \phi \wedge \phi \mid \forall x. \phi \mid [\alpha] \phi \mid [\cdot \cdot] \phi, \end{aligned}$$

where formula ψ is a quantifier-free $FODL_b$ formula in which no modalities $[\cdot]$ and $[\cdot \cdot]$ can appear.

We assume that x is any variable in $\mathbb{Z}$, and f, θ are any function and predicate on $\mathbb{Z}$. We use Var to denote the set of all variables in $FODL_b$, and use Σ to denote the set of all functions and predicates in $FODL_b$. In this paper, as already seen above, we call a $FODL_b$ formula ψ an *arithmetical first-order logical* (AFOL) formula, if in which there exists no modalities $[\cdot]$ and $[\cdot \cdot]$.

The notions of free, bound occurrences of variables and substitution in $FODL_b$ are defined exactly in the same way as in FODL. For more details about them, one can refer to [28]. Informally, a *bound occurrence* of a variable x is in the scope of some quantifier $\forall x$ or $\exists x$, or in the scope of some assignment $x := e$. We call an occurrence *free* if it is not bound. A *substitution* $\phi[e/x]$ replaces in ϕ each free occurrence of x with term e . In this paper we always assume a substitution *admissible*, which means no free occurrences of a variable y in e become bound after the substitution.

In Sect. 4, we have seen that the semantics of PDL_b is given based on a Kripke frame structure $K = (S, m, Re)$, while in $FODL_b$, there is only one set of states S which is determined by the mappings from variables to the domain set.

Definition 6 (States in $FODL_b$). In $FODL_b$, a state (or a “world”) $s: Var \rightarrow \mathbb{Z}$ is a mapping from the set of variables Var to the domain of integers $\mathbb{Z}$. The set of all states is denoted as $\mathbf{S}$.

Table 3
Rules of $FODL_b$.

$\frac{\phi[e/x]}{[x := e]\phi} \quad (x:=e)$	$\frac{\psi \rightarrow \phi}{[\psi?]\phi} \quad (\psi?)$	$\frac{[\alpha][\beta]\phi}{[\alpha; \beta]\phi} \quad ([;])$	$\frac{[\alpha]\phi \wedge [\beta]\phi}{[\alpha \cup \beta]\phi} \quad ([\cup])$	$\frac{\phi \wedge [\alpha][\alpha^*]\phi}{[\alpha^*]\phi} \quad ([*])$
$\frac{\phi \Rightarrow [\alpha]\phi}{\Gamma, \phi \Rightarrow [\alpha^*]\phi, \Delta} \quad (ind)$	$\frac{(v > 0) \wedge \phi(v) \Rightarrow \langle \alpha \rangle \phi(v - 1)}{\Gamma, \exists(v \geq 0). \phi(v) \Rightarrow \langle \alpha^* \rangle \phi(0), \Delta} \quad 1 \text{ (con)}$			
$\frac{\phi \Rightarrow \psi}{\Gamma, [\alpha]\phi \Rightarrow [\alpha]\psi, \Delta} \quad ([gen])$				
(a) rules from FODL				
$\frac{\phi \wedge [b]\phi}{[b]\phi} \quad 2 \text{ ([a])}$	$\frac{[\alpha][\phi \wedge [\alpha][\beta]]\phi}{[\alpha; \beta]\phi} \quad ([;])$	$\frac{[\alpha]\phi \wedge [\beta]\phi}{[\alpha \cup \beta]\phi} \quad ([\cup])$	$\frac{[\alpha^*][[\alpha]]\phi}{[\alpha^*]\phi} \quad ([*])$	
(b) rules special in $FODL_b$				
$\frac{}{\Gamma \Rightarrow \Delta} \quad 3 \text{ (ocl)}$	$\frac{\Gamma \Rightarrow \phi[y/x], \Delta}{\Gamma \Rightarrow \forall x. \phi, \Delta} \quad 4 \text{ (}\forall\text{r)}$	$\frac{\Gamma, \phi[e/x] \Rightarrow \Delta}{\Gamma, \forall x. \phi \Rightarrow \Delta} \quad (\forall\text{f})$		
(c) other rules				

¹ The variable v does not appear in α .

² $b \in \{x := e, \psi?\}$.

³ $\bigwedge_{\phi \in \Gamma} \phi \rightarrow \bigvee_{\psi \in \Delta} \psi$ is a valid AFOL formula.

⁴ y is a new variable with respect to x, ϕ, Γ and Δ .

Therefore, given an interpretation I on all symbols of the signature Σ , the semantics of $FODL_b$ is given by a universal Kripke frame $\mathbf{K}_I = (\mathbf{S}, m_I, Re_I)$ as shown in the following definition.

Definition 7 (Semantics of $FODL_b$ formulas). Given an interpretation I on Σ , $m_I(\alpha)$ and $Re_I(\alpha)$ of $\mathbf{K}_I = (\mathbf{S}, m_I, Re_I)$ are defined as follows:

- $m_I(x := e) =_{df} \{(s, s') \mid s' = s[x \mapsto I_s(e)], s, s' \in \mathbf{S}\}$, where $s[y \mapsto v]$ is defined as

$$s[y \mapsto v](z) =_{df} \begin{cases} v & \text{if } z = y \\ s(z) & \text{otherwise} \end{cases},$$

$I_s(e)$ is inductively defined as:

- (a) $I_s(x) =_{df} s(x)$;
- (b) $I_s(c) =_{df} I(c)$;
- (c) $I_s(f(e_1, \dots, e_n)) =_{df} I(f)(I_s(e_1), \dots, I_s(e_n))$.
- $Re(b) =_{df} m(tt?) \cup m(b)$, b is an assignment $x := e$ or a test $\psi?$.
- $m_I(\alpha)$ and $Re(\alpha)$ for other forms of α are just defined as in Definition 2.

For any formula ϕ , $m_I(\phi)$ is defined as follows:

1. $m_I(\theta(e_1, \dots, e_n)) =_{df} \{s \mid I(\theta)(I_s(e_1), \dots, I_s(e_n)) \text{ is a true relation in the arithmetic theory}\}$.
2. $m_I(\forall x. \phi) =_{df} \{s \mid s \in m_I(\phi[e/x]) \text{ for any } e\}$.
3. $m_I(\phi)$ for other forms of ϕ is just defined as in Definition 2.

From Definition 7, we see that the semantics of $FODL_b$ is largely based on that of PDL_b , except that we need to give an explicit semantics for atomic programs and propositions, and for quantifiers. Below of this paper we always assume an arbitrary interpretation on the signature Σ and simply write m_I and Re_I as m and Re respectively, if no ambiguity is caused.

The satisfaction relation in $FODL_b$ is defined as same as that in PDL_b in Definition 3. So below we use the relation $\models$ for both PDL_b and $FODL_b$ sequents (formulas) and do not distinguish them if unnecessary.

5.2. A proof system of $FODL_b$

We propose a proof system for $FODL_b$. It is proved to be complete relative to AFOL (see Sect. 5.3 below), which is useful for program verification as it can transform a dynamic formula, which captures some property of a program, into one or several AFOL formulas through a proof tree. These AFOL formulas, which are also called *verification conditions*, can then be proved through a deterministic or a semi-deterministic procedure using a SAT/SMT solver.

The proof system of $FODL_b$ is shown in Table 3. The rules of type (a) are inherited from FODL (cf. [28]). We restate them in a form of derivation between sequents. Different from the rules of type (a) in Table 1, rule $(x := e)$ replaces the rules $([\rightarrow])$ and $([\wedge])$, because in FODL rule $(x := e)$ can directly eliminate the dynamic part ' $[x := e]$ ' of a formula $[x := e](\psi \rightarrow \phi)$ (or $[x := e](\phi \wedge \psi)$), without the need to distribute ' $[x := e]$ ' to ψ and ϕ respectively. In FODL, except for the induction rule

Table 4
Other non-primitive rules in DL_b .

$\frac{\Gamma \Rightarrow \psi, \Delta \quad \psi \Rightarrow [\alpha]\psi \quad \psi \Rightarrow \phi}{\Gamma \Rightarrow [\alpha^*]\phi, \Delta} \text{ ([*])}$	
$\frac{\Gamma \Rightarrow \exists v \geq 0. \psi(v), \Delta \quad (v > 0) \wedge \psi(v) \Rightarrow \langle \alpha \rangle \psi(v-1) \quad \psi(0) \Rightarrow \phi}{\Gamma \Rightarrow \langle \alpha^* \rangle \phi, \Delta} \text{ ([*])}$	
$\frac{\Gamma \Rightarrow \phi, \psi, \Delta}{\Gamma \Rightarrow \phi \vee \psi, \Delta} (\vee r)$	$\frac{\Gamma, \phi \Rightarrow \Delta \quad \Gamma, \psi \Rightarrow \Delta}{\Gamma, \phi \vee \psi \Rightarrow \Delta} (\vee l)$
$\frac{\Gamma \Rightarrow \phi[e/x], \Delta}{\Gamma \Rightarrow \exists x. \phi, \Delta} (\exists r)$	$\frac{\Gamma, \phi[y/x] \Rightarrow \Delta}{\Gamma, \exists x. \phi \Rightarrow \Delta} 1 (\exists l)$
$\frac{\Gamma, \phi \Rightarrow \psi, \Delta}{\Gamma \Rightarrow \phi \rightarrow \psi, \Delta} (\rightarrow r)$	$\frac{\Gamma \Rightarrow \phi, \Delta \quad \Gamma, \psi \Rightarrow \Delta}{\Gamma, \phi \rightarrow \psi \Rightarrow \Delta} (\rightarrow l)$

(*ind*), a *convergence* rule is needed for inductive proof of a dynamic formula of the form $\langle \alpha^* \rangle \phi$. Rule (*con*) has a similar meaning as (*ind*) but has a variable v indicating the termination of program α^* in finite steps.

In practical verifications, the rules ([*]) and ([*]) of Table 4 are often applied for decomposing a star program α^* . They are known as the *invariant rules*, where formula ψ is often called a *loop invariant* of α^* . Rules ([*]) and ([*]) are not primitive and can be derived from (*ind*) and (*con*) using ([*gen*]), we omit the proofs here and one can refer to e.g. [8] for more details.

The rules of type (b) are almost the same as those in PDL_b except for program b , which can be an assignment.

The oracle rule of type (c) is an axiom, it terminates a proof branch when the sequent $\Gamma \Rightarrow \Delta$ is a valid AFOL formula, i.e., when there is no dynamic formulas or subformulas in the sequent. Rules ($\forall r$) and ($\forall l$) are rules in first-ordered logic for introducing the quantifier $\forall$. These two rules are necessary for eliminating the quantifier in first-ordered logic during a proof. Note that the rules for the quantifier $\exists$ are not primitive and are listed in Table 4.

Similar as the deduction relation for PDL_b defined in Definition 4, a sequent ν can be proved in $FODL_b$, denoted by $\vdash \nu$, if it can be derived by applying rules in Table 3 and Table 2.

The proof system of $FODL_b$ is also sound, which is a direct result from the soundness of PDL_b and FODL and according to the semantics of $FODL_b$.

Theorem 4 (Soundness of $FODL_b$). *Given a $FODL_b$ formula ϕ , if $\vdash \phi$, then $\models \phi$.*

For the rules of type (a), their soundness directly follows the soundness of FODL. The soundness of the rules of type (c) is trivial and from classical first-ordered logic. As for the rules of type (b), their soundness is already proved in PDL_b in Appendix A, where the only difference is that in $FODL_b$, program b can be an assignment. We omit the proof of Theorem 4 in this paper.

To close this section, we give a simple example showing how to derive a dynamic formula in $FODL_b$. This is a good example to see how a practical verification of a program property can be carried out in our proposed $FODL_b$. We consider a first-order version of the regular program α in Example 1, where let

$$\begin{aligned} \beta &= x := 10 \cup x := 5 \cup (ff?; x := 1), \\ \gamma &= x \geq 10?; y := 1; (y := 1)^*; y := 0. \end{aligned}$$

Compared to Example 1, we have $\phi = ff$, $\psi = (x \geq 10)$. Actions a, a_1, a_2 are replaced by $x := 10, x := 5, x := 1$ respectively. Action b is replaced by $y := 0$, while action c is replaced by $y := 1$. We prove the dynamic formula

$$x = 0 \rightarrow [\beta; \gamma](x > 0 \rightarrow x \geq 5)$$

is true, which is a safety property checking that starting from state s_1 in Fig. 1 satisfying $x = 0$, whether each state of α satisfies $x > 0 \rightarrow x \geq 5$.

Fig. 2 shows the procedure of the proof. Starting from the root sequent (1), it is successfully derived when all leaf nodes of the proof tree terminate with applications of rule (*ocl*) or rule (*ax*). Note that rules of the form ($X \downarrow$) come from Proposition 9 of Appendix B. Due to the limit of space, we only consider one derivation path, which is to the leaf node (23) in the proof tree, while ignoring all other branches by ‘...’. At node (15), a loop invariant tt is chosen when applying rule ([*]).

5.3. Relative completeness of $FODL_b$

In this paper, we mainly discuss $FODL_b$ ’s completeness relative to AFOL, because generally, any predicate logic that subsumes Peano’s arithmetic theory is incomplete, due to Gödel’s incompleteness theorem [31].

Similar as in PDL_b , we rely on the “equivalent capability” between the proof system of $FODL_b$ and that of FODL to give a proof. This proof is much simpler than directly applying the methodology for proving the relative completeness of FODL (cf. [29]).

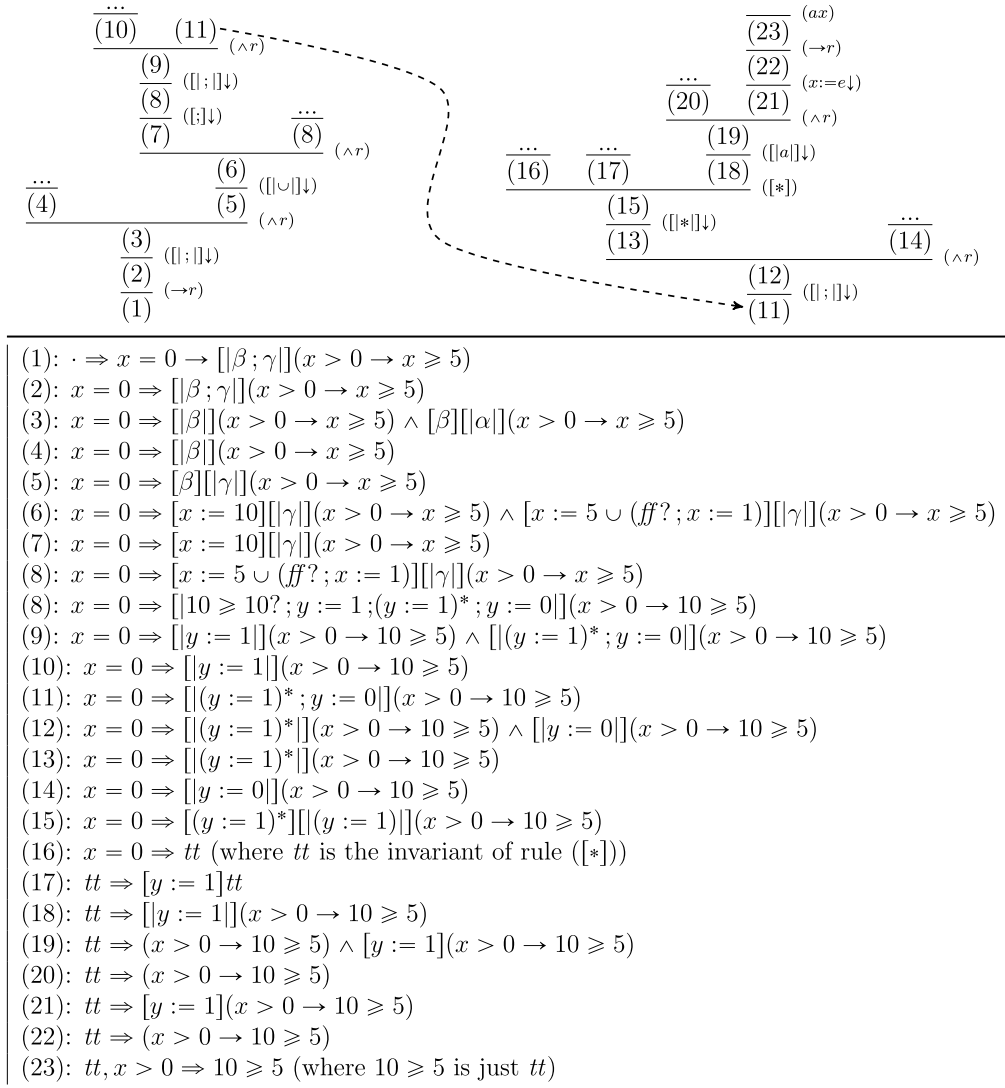


Fig. 2. The derivations of formula $x = 0 \rightarrow [[\beta; \gamma]](x > 0 \rightarrow x \geq 5)$.

From Proposition 2 and Proposition 4, we can see their proofs are actually only related to the proof rules special for PDL_b , i.e., the rules of type (b) in Table 1. Therefore, it is not hard to see that these two propositions can also be adapted to $FODL_b$, since the proof rules special for $FODL_b$ are nearly the same as those of PDL_b , while the only difference is that in $FODL_b$, a program b can be an assignment. Therefore, we have the following proposition.

Proposition 5. For any $FODL_b$ formula ϕ , there is a $FODL$ formula ψ such that

$$\models \phi \leftrightarrow \psi \text{ and } \vdash \phi \leftrightarrow \psi.$$

The proof of $\vdash \phi \leftrightarrow \psi$ in Proposition 5 is similar as that of Proposition 4, where the only difference is the treatment of rule $([a])$ when program b is an assignment. Similar for $\models \phi \leftrightarrow \psi$, whose proof is similar to Proposition 2. We omit their proofs in this paper.

Based on Proposition 5, we show that the proof system of $FODL_b$ is complete relative to AFOL, which relies on the relative completeness of FODL proved in [29], stated as follows.

Proposition 6 (Relative completeness of FODL [29]). In the proof system consisting of all the rules of Table 3 and 2 and all axioms of tautologies in AFOL, for any FODL formula ψ , $\models \psi$ implies $\vdash \psi$.

Theorem 5 is straightforward from Proposition 5 and Proposition 6. Its proof is similar to Theorem 3.

Theorem 5 (Relative completeness of $FODL_b$). *If all tautologies in AFOL are derived, for any $FODL_b$ formula ϕ , $\models \phi$ implies $\vdash \phi$.*

Proof. If $\models \phi$, by Proposition 5 there exists a FODL formula ψ such that $\vdash \phi \leftrightarrow \psi$ and $\vdash \phi \Leftrightarrow \psi$. So $\models \psi$. From Proposition 6, if all tautologies in AFOL are derived, we have $\vdash \psi$. By rule (cut) we can derive $\vdash \phi$ from $\vdash \phi \Leftrightarrow \psi$ and $\vdash \psi$. $\square$

6. Comparison between $FODL_b$ and DLT

In this section, we compare $FODL_b$ with DLT. As already mentioned in Sect. 1, we show that $FODL_b$ is a natural extension of DLT and thus it subsumes DLT as its sub-logic. Essentially, this is because branching modality actually captures the same set of safety properties as the trace modality for those programs where there are no aborting traces. In other words, they only differs from each other when concerning aborting traces. For the same reason, as one of the main motivations of proposing the branching modality, in this section, we show that the rule ($\llbracket \cdot \rrbracket$) of DLT – the corresponding rule of rule ($\llbracket \cdot \rrbracket$) – does not hold for regular programs in $FODL_b$.

Below we first introduce the trace semantics of DLT, based on which we compare the expressiveness of $FODL_b$ and DLT. At the end, we discuss why the branching modality is chosen for our proposed logic.

6.1. Trace semantics of DLT

A trace is a finite or infinite sequence of states, denoted by $s_1 s_2 \dots s_n \dots$. We use tr^i to denote the i th state of trace tr , $i \geq 0$. We use tr^b to denote the first state of trace tr , so $tr^0 = tr^b$, and use tr^e to denote the last state of trace tr provided that tr is finite.

Given two traces tr_1, tr_2 , the concatenation between tr_1 and tr_2 is a partial function, denoted by $tr_1 \cdot tr_2$, and given as follows:

$$tr_1 \cdot tr_2 =_{df} \begin{cases} tr_1 & \text{if } tr_1 \text{ is infinite,} \\ tr_1^0 tr_1^1 \dots tr_1^e tr_2^1 \dots tr_2^n \dots & \text{if } tr_1 \text{ is finite and } tr_1^e = tr_2^b \end{cases}.$$

We lift $\cdot$ to an operator between two sets of traces. Given two sets of traces T_1 and T_2 , $T_1 \cdot T_2$ is defined as

$$T_1 \cdot T_2 =_{df} \{tr_1 \cdot tr_2 \mid tr_1 \in T_1, tr_2 \in T_2\}.$$

Note that when concatenating two traces, the first element of the second trace (i.e. tr_2^b above) is abandoned.

The semantics of DLT [17] is given as a Kripke frame $(\mathbf{S}, m_{tr})$, where $m_{tr}(\alpha)$ maps a program α to a set of traces, inductively defined as follows:

- $m_{tr}(x := e) =_{df} \{ss' \mid s' = s[x \mapsto I_s(e)], s, s' \in \mathbf{S}\}$, where $s[y \mapsto v]$ is defined as

$$s[y \mapsto v](z) =_{df} \begin{cases} v & \text{if } z = y \\ s(z) & \text{otherwise} \end{cases};$$

- $m_{tr}(\alpha; \beta) =_{df} m_{tr}(\alpha) \cdot m_{tr}(\beta)$;
- $m_{tr}(\text{if } \psi \text{ then } \alpha \text{ else } \beta) =_{df} \{tr \mid tr^b \models \psi, tr \in m_{tr}(\alpha)\} \cup \{tr \mid tr^b \models \neg\psi, tr \in m_{tr}(\beta)\}$;
- $m_{tr}(\text{while } \psi \text{ do } \alpha) =_{df} T_1 \cup T_2 \cup T_3$, where
 - (i) $T_1 =_{df} \{ss \mid s \models \neg\psi\} \cup \{tr_1 \cdot \dots \cdot tr_n \mid tr_i \in m_{tr}(\alpha) \text{ is finite and } tr_i^b \models \psi \text{ for all } i \geq 0, tr_n^e \models \neg\psi\}$;
 - (ii) $T_2 =_{df} \{tr_1 \cdot tr_2 \cdot \dots \mid tr_i \in m_{tr}(\alpha) \text{ is finite and } tr_i^b \models \psi \text{ for all } i \geq 0\}$;
 - (iii) $T_3 =_{df} \{tr_1 \cdot \dots \cdot tr_n \mid tr_i \in m_{tr}(\alpha) \text{ is finite and } tr_i^b \models \psi \text{ for all } 1 \leq i < n, tr_n \in m_{tr}(\alpha) \text{ is infinite and } tr_n^b \models \psi\}$.

$m_{tr}(\varphi)$ for any formula φ is defined as follows:

1. $m_{tr}(tt) =_{df} \mathbf{S}$;
2. $m_{tr}(\neg\phi) =_{df} \mathbf{S} - m_{tr}(\phi)$;
3. $m_{tr}(\phi \wedge \psi) =_{df} m_{tr}(\phi) \cap m_{tr}(\psi)$;
4. $m_{tr}([\alpha]\phi) =_{df} \{s \mid \text{for all } tr \in m_{tr}(\alpha) \text{ with } tr^b = s \text{ and } tr^e = s' \text{ if } tr^e \text{ exists}\}$;
5. $m_{tr}(\llbracket \alpha \rrbracket \phi) =_{df} \{s \mid \text{for all } tr \text{ with } tr^b = s, tr(i) \in m_{tr}(\phi) \text{ for all } i \geq 0\}$.

The traces of *while...do...* statement are classified into 3 categories. T_1 consists of all finite traces generated by *while* loops that will eventually terminate. T_2 consists of all traces generated by non-terminating *while* loops, while each segment of a trace of the loop body is finite. T_3 consists of all infinite traces where each segment of the loop body is finite, except for the last segment. For more explanations one can see [17].

6.2. Expressibility of DLT in $FODL_b$

The main difference between the semantics of DLT and the semantics of $FODL_b$ is that the semantics of a *while* program α is a set of traces. As a result, the semantics of $\llbracket \alpha \rrbracket \phi$ is about the set of traces of α where each state satisfies formula ϕ . However, since each *while* program either executes eternally or terminates without abortion, $\llbracket \alpha \rrbracket \phi$ actually means the same thing as $\llbracket \alpha \rrbracket \phi$ in $FODL_b$. Because intuitively, each state reachable from α is a state of a (non-aborting) trace of α . From this observation, we conclude that $FODL_b$ subsumes DLT, as stated in the following proposition.

Proposition 7 (Expressibility of DLT in $FODL_b$). *For any DLT formula ϕ , there exists a $FODL_b$ formula ψ such that*

$$\models \phi \leftrightarrow \psi.$$

The proof is given in Appendix C. As DLT and $FODL_b$ share the same AFOL part, the critical part of the proof is to show that given a DLT formula $\llbracket \alpha \rrbracket \phi^b$ where ϕ^b is a AFOL formula, whether there exists a regular program β such that $\models \llbracket \alpha \rrbracket \phi^b \leftrightarrow \llbracket \beta \rrbracket \phi^b$ (see condition (C.1) in the proof of Proposition 7 in Appendix C). Lemma 2 and Lemma 3 are auxiliary propositions for proving Proposition 7. Lemma 2 states the “non-aborting” property possessed by *while* programs, while Lemma 3 links reachable states of program β with states on traces of α . Along the proof of Lemma 3, a transformation from *while* programs to regular programs is built. The main idea behind Lemma 3 is that although the semantics of DLT statements contains infinite traces, we can still build a correspondence between the states of traces of *while* programs and the reachable states of regular programs in $FODL_b$. More details are given in Appendix C.

6.3. The extension of rule $(\llbracket \cdot \rrbracket)$ for regular programs — why it doesn't work

From Sect. 6.1 and 6.2, we have seen the difference between trace and branching modality. One may ask that whether the proof system of $FODL_b$ in Sect. 5.2 also applies if we take the trace modality rather than the branching one. In other words, can we directly extend the *while* programs of DLT into regular programs with the trace semantics in a natural way? In the following, we show that rule $(\llbracket \cdot \rrbracket)$ of DLT:

$$\frac{\llbracket \alpha \rrbracket \phi \wedge \llbracket \alpha \rrbracket \llbracket \beta \rrbracket \phi}{\llbracket \alpha ; \beta \rrbracket \phi} (\llbracket \cdot \rrbracket),$$

which corresponds the rule $(\llbracket \cdot \rrbracket)$ in $FODL_b$, is not sound when applied to regular programs with tests. That is one of the main reasons we adopt the proposed branching modality in our DL_b .

The trace semantics of regular programs can be obtained from the trace semantics of *while* programs in a natural way. Given a regular program α , its trace semantics, denoted by $Tr(\alpha)$, is inductively defined as:

- $Tr(\alpha \cup \beta) =_{df} Tr(\alpha) \cup Tr(\beta)$;
- $Tr(\alpha^*) =_{df} \bigcup_{n \geq 0} Tr(\alpha^n)$;
- $Tr(\gamma)$ for a program γ of other forms are defined as $m_{tr}(\gamma)$ in Sect. 6.1.

Tr inherits the definition of $m_{tr}(\alpha; \beta)$, so Tr does not include aborting traces of regular programs. Because a trace of α can be aborted by a test in β , so that it is not a trace of the whole program $\alpha; \beta$. As already shown in Example 1, if we adapt trace modality $\llbracket \cdot \rrbracket$ to regular programs by defining

$$m(\llbracket \alpha \rrbracket \phi) =_{df} \{s \mid \text{for all } tr \in Tr(\alpha), tr^b = s \text{ and } tr^i \in m(\phi)\},$$

$\llbracket \alpha \rrbracket \phi$ would capture a different property than $\llbracket \alpha \rrbracket \phi$. For this reason, trace modality $\llbracket \cdot \rrbracket$ is not congruent to the compositionality of regular programs.

Consider a safety property $\llbracket \beta; \gamma \rrbracket (x > 0 \rightarrow x \geq 10)$ for the program α defined in Sect. 5.2, given $x = 0$, according to $m(\llbracket \alpha \rrbracket \phi)$ defined above, $\llbracket \beta; \gamma \rrbracket (x > 0 \rightarrow x \geq 10)$ is true. However, $\llbracket \beta \rrbracket (x > 0 \rightarrow x \geq 10)$ does not hold for program β . Because in β the branch $x := 5$ can be executed, which makes formula $x > 0 \rightarrow x \geq 10$ unsatisfied.

As a consequence, in this example, we see that the premise of rule $(\llbracket \cdot \rrbracket)$ becomes too strong, so that $\llbracket \beta; \gamma \rrbracket (x > 0 \rightarrow x \geq 10)$ does not imply $\llbracket \beta \rrbracket (x > 0 \rightarrow x \geq 10)$. In other words, the sequent $\llbracket \beta; \gamma \rrbracket (x > 0 \rightarrow x \geq 10) \Rightarrow \llbracket \beta \rrbracket (x > 0 \rightarrow x \geq 10) \wedge \llbracket \gamma \rrbracket (x > 0 \rightarrow x \geq 10)$ is not valid.

Generally, in regular programs, a dynamic formula $\llbracket \alpha \rrbracket \phi$ expresses a ‘stronger’ safety property than $\llbracket \alpha \rrbracket \phi$, since $\llbracket \alpha \rrbracket \phi$ only requires those terminating traces satisfy ϕ . Therefore, rule $(\llbracket \cdot \rrbracket)$ is no longer suitable for regular programs. To find an equivalent logical assertion for a formula $\llbracket \alpha; \beta \rrbracket \phi$ that dissolves a program $\alpha; \beta$, one needs to filter out those aborting traces in α , so that proving “each state of α that leads to terminations satisfies ϕ ” is possible. It could be interesting to ask whether such an assertion exists.

7. Conclusion and future work

In this paper, we mainly propose a novel temporal dynamic logic DL_b that merges dynamic logic and temporal logic. The first-ordered level of DL_b — $FODL_b$ — is useful for specification and verification of safety properties of regular programs in practice. DL_b extends traditional dynamic logic with a branching modality $[\cdot \cdot]$ and adopts a semantics of reachable states Re , which does not exclude aborting traces of regular programs. Such a characteristic of branching modality $[\cdot \cdot]$ allows correctly specifying safety properties for regular programs in which all intermediate states are required to satisfy a formula. More importantly, its semantics is congruent to the compositionality of regular programs, which thus makes the direct extension of DLT's proof system to $FODL_b$'s possible. We build the theory of DL_b on both propositional and first-ordered levels, defining two logics PDL_b and $FODL_b$, and build their proof systems respectively. We analyze the decidability, soundness and (relative) completeness of PDL_b and $FODL_b$ by comparing their expressiveness and deduction capabilities with their counterparts PDL and FODL in a formal way. In the end, we make a comparison between DLT and $FODL_b$ and show that despite different semantics adopted, $FODL_b$ subsumes DLT in terms of expressiveness.

As seen in this paper, we actually consider the semantics of regular programs as the combination of m and Re . One of our future work will focus on digging up the connection between it and the *graph semantics* of regular programs. Graph semantics was proposed in [32], whose equivalence relation exactly captures the bisimulation relation [33]. One interesting problem is that whether graph semantics is equivalent to the combinational semantics of m and Re . Another future work is to extend DL_b or $FODL_b$ with modalities to express a larger set of temporal properties, such as liveness properties. One way is to introduce temporal formulas, such as $\phi \mathcal{U} \psi$, into $DL_b/FODL_b$, as what has been done in dynamic trace logic [25]. In such a logic, looking for a sound and (relatively) complete proof system would be the primary challenge.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgements

We thank Prof. Xinxin Liu for his insightful suggestions towards this paper.

This work is partially supported by the Youth Project of National Science Foundation of China (No. 62102329), the Project of National Science Foundation of Chongqing (No. cstc2021jcyj-bshX0120), the Capacity Development Grant of Southwest University (No. SWU116007) and the Key Projects of National Science Foundation of China (No. 61732019, No. 62032019).

Appendix A. Proof for the soundness of PDL_b

For the soundness of PDL_b , we only need to prove the following proposition.

Proposition 8 (Soundness of the rules of type (b)). Rules $([a];)$, $([\cdot];)$, $([\cdot \cup \cdot])$ and $([\cdot * \cdot])$ of Table 1 are sound.

Proof. By the meaning of rule $\frac{\phi}{\psi}$, it is equivalent to prove that $\phi \leftrightarrow \psi$ is valid.

In the following proofs for different rules, let s be any state of S in any Kripke frame K .

For rule $([a];)$, $s \models [[a]]\phi$ iff $s' \models \phi$ for all $(s, s') \in Re(b)$, iff $s \models \phi$ as $(s, s) \in m(tt?) \subseteq Re(b)$ and $s' \models \phi$ for all $(s, s') \in m(b) \subseteq Re(b)$, iff $s \models \phi$ and $s \models [b]\phi$, which is $s \models \phi \wedge [b]\phi$.

For rule $([\cdot];)$, $s \models [\alpha; \beta]\phi$ iff $s' \models \phi$ for all $(s, s') \in Re(\alpha; \beta)$, iff

$$s' \models \phi \text{ for all } (s, s') \in Re(\alpha) \text{ and } (s, s') \in m(\alpha) \circ Re(\beta). \quad (A.1)$$

Note that for any $(s, s') \in m(\alpha) \circ Re(\beta)$, there exists a s'' such that $(s, s'') \in m(\alpha)$ and $(s'', s') \in Re(\beta)$. So $s' \models \phi$ for all $(s, s') \in m(\alpha) \circ Re(\beta)$ iff $s' \models \phi$ for all $(s, s'') \in m(\alpha)$ and $(s'', s') \in Re(\beta)$, iff $s'' \models [\beta]\phi$ for all $(s, s'') \in m(\alpha)$, iff $s \models [\alpha][[\beta]]\phi$. Therefore, (A.1) holds iff $s \models [\alpha][[\beta]]\phi$ and $s \models [\alpha][[\beta]]\phi$, which is $s \models [\alpha][[\beta]]\phi \wedge [\alpha][[\beta]]\phi$.

For rule $([\cdot \cup \cdot])$, $s \models [\alpha \cup \beta]\phi$ iff $s' \models \phi$ for all $(s, s') \in Re(\alpha \cup \beta)$, iff $s' \models \phi$ for all $(s, s') \in Re(\alpha)$ and $s' \models \phi$ for all $(s, s') \in Re(\beta)$, iff $s \models [\alpha]\phi$ and $s \models [\beta]\phi$, which is $s \models [\alpha]\phi \wedge [\beta]\phi$.

For rule $([\cdot * \cdot])$, we first prove by induction that $Re(\alpha^n) = \bigcup_{i=0}^{n-1} m(\alpha^i) \circ Re(\alpha)$ on $n \geq 1$. For the basic case when $n = 1$, we have $m(\alpha^0) \circ Re(\alpha) = m(tt?) \circ Re(\alpha) = Re(\alpha)$. For the inductive case, we have that

$$\begin{aligned}
Re(\alpha^{n+1}) &= Re(\alpha ; \alpha^n) = Re(\alpha) \cup m(\alpha) \circ Re(\alpha^n) \\
&= Re(\alpha) \cup m(\alpha) \circ ((\bigcup_{i=0}^{n-1} m(\alpha^i)) \circ Re(\alpha)) && \text{(by inductive hypothesis)} \\
&= (m(\alpha^0) \cup m(\alpha) \circ (\bigcup_{i=0}^{n-1} m(\alpha^i))) \circ Re(\alpha) && \text{(by } m(\alpha^0) \circ Re(\alpha) = Re(\alpha) \text{)} \\
&= (\bigcup_{i=0}^n m(\alpha^i)) \circ Re(\alpha).
\end{aligned}$$

In addition, when $n \geq 1$, we have

$$\begin{aligned}
\bigcup_{n \geq 1} Re(\alpha^n) &= \bigcup_{n \geq 1} (\bigcup_{i=0}^{n-1} m(\alpha^i)) \circ Re(\alpha) \\
&= (\bigcup_{n \geq 1} \bigcup_{i=0}^{n-1} m(\alpha^i)) \circ Re(\alpha) \\
&= (\bigcup_{i=0}^{\infty} m(\alpha^i)) \circ Re(\alpha).
\end{aligned} \tag{A.2}$$

In one hand, if $s \models [\alpha^*] \phi$, then $s' \models \phi$ for all $(s, s') \in Re(\alpha^*) = \bigcup_{n \geq 0} Re(\alpha^n)$. To show $s \models [\alpha^*][\alpha] \phi$, we prove that for any $(s, u) \in (\bigcup_{i \geq 0} m(\alpha^i)) \circ Re(\alpha)$, $u \models \phi$. By equation (A.2) we have that $(s, u) \in \bigcup_{n \geq 1} Re(\alpha^n) \subseteq \bigcup_{n \geq 0} Re(\alpha^n) = Re(\alpha^*)$, so $u \models \phi$. On the other hand, if $s \models [\alpha^*][\alpha] \phi$, then $s' \models \phi$ for all $(s, s') \in (\bigcup_{i \geq 0} m(\alpha^i)) \circ Re(\alpha)$. To show $s \models [\alpha^*] \phi$, we prove that for any $(s, u) \in Re(\alpha^*) = \bigcup_{n \geq 0} Re(\alpha^n)$, $u \models \phi$. If $(s, u) \in Re(\alpha^0)$, then $u = s$ since $Re(\alpha^0) = m(tt?)$. Then it is clear to see that $(s, u) \in m(tt?) \circ m(tt?) = m(\alpha^0) \circ m(tt?) \subseteq m(\alpha^0) \circ Re(\alpha) \subseteq (\bigcup_{i \geq 0} m(\alpha^i)) \circ Re(\alpha)$, so $u \models \phi$. If $(s, u) \in \bigcup_{n \geq 1} Re(\alpha^n)$, then by equation (A.2) we directly get that $(s, u) \in (\bigcup_{i \geq 0} m(\alpha^i)) \circ Re(\alpha)$, hence $u \models \phi$. $\square$

Appendix B. Proof for the completeness of PDL_b

We first prove Proposition 4. The following two propositions serve for the proof of Proposition 4.

Lemma 1. *The following rules are derivable from the proof system of PDL_b :*

1. $\frac{\Gamma \Rightarrow \phi, \Delta}{\Gamma \Rightarrow \phi, \psi, \Delta} \text{ (w } r \text{)};$
2. $\frac{\phi \Rightarrow \psi}{\Gamma, \phi \Rightarrow \psi, \Delta} \text{ (ctx)}.$

Proof. Rule (w r) can be derived through:

$$\frac{\frac{\frac{\Gamma \Rightarrow \phi, \Delta}{\Gamma, \neg \phi \Rightarrow \Delta} \text{ (}\neg l\text{)}}{\Gamma, \neg \phi, \neg \psi \Rightarrow \Delta} \text{ (w)}}{\frac{\Gamma, \neg \phi \Rightarrow \psi, \Delta}{\Gamma \Rightarrow \phi, \psi, \Delta} \text{ (}\neg r\text{)}} \text{ (}\neg r\text{)},$$

while rule (ctx) can be derived by applying rule (w) and (w, r) for several times on both sides of the sequent $\Gamma, \phi \Rightarrow \psi, \Delta$ until only ϕ and ψ are left. $\square$

Proposition 9. *If $\frac{\phi}{\psi} \text{ (x)}$ is a rule (with name (x)), then $\frac{\Gamma \Rightarrow \phi, \Delta}{\Gamma \Rightarrow \psi, \Delta} \text{ (x } \downarrow \text{)}$ and $\frac{\Gamma, \psi \Rightarrow \Delta}{\Gamma, \phi \Rightarrow \Delta} \text{ (x } \uparrow \text{)}$ (for any contexts Γ and Δ) are derivable from the proof system of PDL_b .*

In the paper, sometimes we use $(x \uparrow)$ and $(x \downarrow)$ to represent the rules $(\uparrow)$ and $(\downarrow)$ of rule (x). For example, $([a], \uparrow)$.

Proof of Proposition 9. Through the following derivations, we see that $\frac{\Gamma \Rightarrow \phi, \Delta}{\Gamma \Rightarrow \psi, \Delta}$ is derivable:

$$\frac{\frac{\Gamma \Rightarrow \phi, \Delta}{\Gamma \Rightarrow \phi, \psi, \Delta} (w\ r) \quad \frac{\phi \Rightarrow \psi}{\Gamma, \phi \Rightarrow \psi, \Delta} (ctx)}{\Gamma \Rightarrow \psi, \Delta} (cut).$$

Similarly, the soundness of $\frac{\Gamma, \psi \Rightarrow \Delta}{\Gamma, \phi \Rightarrow \Delta}$ is guaranteed by the following derivations:

$$\frac{\frac{\phi \Rightarrow \psi}{\Gamma, \phi \Rightarrow \psi, \Delta} (ctx) \quad \frac{\Gamma, \psi \Rightarrow \Delta}{\Gamma, \phi, \psi \Rightarrow \Delta} (w)}{\Gamma, \phi \Rightarrow \Delta} . \quad \square$$

Proof of Proposition 4. To prove $\vdash \phi \Leftrightarrow \psi$, it means to prove both $\vdash \phi \Rightarrow \psi$ and $\vdash \psi \Rightarrow \phi$. In other words, we prove that the sequents $\phi \Rightarrow \psi$ and $\psi \Rightarrow \phi$ can be derived according to the rules in Table 1 and 2. We proceed by simultaneous induction on the syntactic structure of PDL_b formulas and regular programs.

If $\phi = \neg\varphi$, by induction on the structure of PDL_b formulas there exists a PDL formula ψ such that $\vdash \varphi \Leftrightarrow \psi$. From the sequents $\varphi \Rightarrow \psi$ and $\psi \Rightarrow \varphi$, we have the following derivations:

$$\frac{\frac{\psi \Rightarrow \varphi}{\psi, \neg\varphi \Rightarrow \cdot} (\neg l) \quad \frac{\varphi \Rightarrow \psi}{\varphi, \neg\psi \Rightarrow \cdot} (\neg l)}{\neg\varphi \Rightarrow \neg\psi} (\neg r) \quad \text{and} \quad \frac{\frac{\varphi \Rightarrow \psi}{\varphi, \neg\psi \Rightarrow \cdot} (\neg l) \quad \frac{\psi \Rightarrow \varphi}{\neg\psi \Rightarrow \neg\varphi} (\neg r)}{\neg\psi \Rightarrow \neg\varphi} .$$

Therefore, $\vdash \neg\varphi \Leftrightarrow \neg\psi$. Since ψ is a PDL formula, so is $\neg\psi$.

If $\phi = \varphi_1 \wedge \varphi_2$, by induction on the structure of PDL_b formulas there are PDL formulas ψ_1 and ψ_2 such that $\vdash \varphi_1 \Leftrightarrow \psi_1$ and $\vdash \varphi_2 \Leftrightarrow \psi_2$. We have the following derivations:

$$\frac{\frac{\varphi_1 \Rightarrow \psi_1}{\varphi_1, \varphi_2 \Rightarrow \psi_1} (w) \quad \frac{\varphi_2 \Rightarrow \psi_2}{\varphi_1, \varphi_2 \Rightarrow \psi_2} (w)}{\varphi_1 \wedge \varphi_2 \Rightarrow \psi_1} (\wedge l) \quad \frac{\frac{\varphi_2 \Rightarrow \psi_2}{\varphi_1, \varphi_2 \Rightarrow \psi_2} (w) \quad \frac{\varphi_1 \Rightarrow \psi_1}{\varphi_1 \wedge \varphi_2 \Rightarrow \psi_2} (w)}{\varphi_1 \wedge \varphi_2 \Rightarrow \psi_2} (\wedge l) \quad \text{and} \quad \frac{\frac{\psi_1 \Rightarrow \varphi_1}{\psi_1, \psi_2 \Rightarrow \varphi_1} (w) \quad \frac{\psi_2 \Rightarrow \varphi_2}{\psi_1, \psi_2 \Rightarrow \varphi_2} (w)}{\psi_1 \wedge \psi_2 \Rightarrow \varphi_1} (\wedge l) \quad \frac{\frac{\psi_2 \Rightarrow \varphi_2}{\psi_1, \psi_2 \Rightarrow \varphi_2} (w) \quad \frac{\psi_1 \Rightarrow \varphi_1}{\psi_1 \wedge \psi_2 \Rightarrow \varphi_2} (w)}{\psi_1 \wedge \psi_2 \Rightarrow \varphi_2} (\wedge l) .$$

Therefore, $\vdash \varphi_1 \wedge \varphi_2 \Leftrightarrow \psi_1 \wedge \psi_2$. Since ψ_1 and ψ_2 are PDL formulas, so is $\psi_1 \wedge \psi_2$.

When $\phi = [|\alpha|]\varphi$, consider the following cases of α .

If $\alpha = a$ for any atomic program a , by induction on the structure of PDL_b formulas there exists a PDL formula ψ such that $\vdash \varphi \Leftrightarrow \psi$. We have the following derivations:

$$\frac{\frac{\varphi \Rightarrow \psi}{\varphi, [a]\varphi \Rightarrow \psi} (w) \quad \frac{\frac{\varphi \Rightarrow \psi}{\varphi, [a]\varphi \Rightarrow \psi} (w) \quad \frac{\psi \Rightarrow \varphi}{[a]\psi \Rightarrow [a]\varphi} (gen)}{\varphi \wedge [a]\varphi \Rightarrow \psi} (\wedge l) \quad \frac{\frac{\varphi \Rightarrow \psi}{\varphi, [a]\varphi \Rightarrow \psi} (w) \quad \frac{\psi \Rightarrow \varphi}{[a]\psi \Rightarrow [a]\varphi} (gen)}{\varphi \wedge [a]\varphi \Rightarrow \psi} (\wedge l) \quad \frac{\frac{\varphi \wedge [a]\varphi \Rightarrow \psi}{[a]\varphi \Rightarrow \psi} ([a] \uparrow) \quad \frac{\frac{\varphi \wedge [a]\varphi \Rightarrow \psi}{[a]\varphi \Rightarrow \psi} ([a] \uparrow) \quad \frac{\psi \Rightarrow \varphi}{[a]\psi \Rightarrow [a]\varphi} (gen)}{[a]\varphi \Rightarrow \psi} (\wedge r) \quad \frac{\frac{\varphi \wedge [a]\varphi \Rightarrow \psi}{[a]\varphi \Rightarrow \psi} ([a] \uparrow) \quad \frac{\psi \Rightarrow \varphi}{[a]\psi \Rightarrow [a]\varphi} (gen)}{[a]\varphi \Rightarrow \psi} (\wedge r) .$$

and

$$\frac{\frac{\psi \Rightarrow \varphi}{\psi, [a]\psi \Rightarrow \varphi} (w) \quad \frac{\frac{\psi \Rightarrow \varphi}{\psi, [a]\psi \Rightarrow \varphi} (w) \quad \frac{\psi \Rightarrow \varphi}{[a]\psi \Rightarrow [a]\varphi} (gen)}{\psi \wedge [a]\psi \Rightarrow \varphi} (\wedge l) \quad \frac{\frac{\psi \Rightarrow \varphi}{\psi, [a]\psi \Rightarrow \varphi} (w) \quad \frac{\psi \Rightarrow \varphi}{[a]\psi \Rightarrow [a]\varphi} (gen)}{\psi \wedge [a]\psi \Rightarrow \varphi} (\wedge l) \quad \frac{\frac{\psi \wedge [a]\psi \Rightarrow \varphi}{[a]\psi \Rightarrow \varphi} ([a] \downarrow) \quad \frac{\psi \wedge [a]\psi \Rightarrow \varphi}{[a]\psi \Rightarrow \varphi} ([a] \downarrow)}{\psi \wedge [a]\psi \Rightarrow \varphi} (\wedge r) .$$

So we have $\vdash [a]\varphi \Leftrightarrow \psi \wedge [a]\psi$. Since ψ is a PDL formula, so is $\psi \wedge [a]\psi$.

If $\alpha = \alpha_1; \alpha_2$, by induction on the structure of regular programs, there are PDL formulas ψ_1 and ψ_2 such that $\vdash [|\alpha_1|]\varphi \Leftrightarrow \psi_1$ and $\vdash [|\alpha_2|]\varphi \Leftrightarrow \psi_2$. We can have the following derivations:

$$\begin{array}{c}
\frac{[\alpha_1]\varphi \Rightarrow \psi_1}{[\alpha_1]\varphi, [\alpha_1][\alpha_2]\varphi \Rightarrow \psi_1} (w) \quad \frac{[\alpha_2]\varphi \Rightarrow \psi_2}{[\alpha_1][\alpha_2]\varphi \Rightarrow [\alpha_1]\psi_2} (gen) \\
\frac{[\alpha_1]\varphi, [\alpha_1][\alpha_2]\varphi \Rightarrow \psi_1}{[\alpha_1]\varphi \wedge [\alpha_1][\alpha_2]\varphi \Rightarrow \psi_1} (\wedge l) \quad \frac{[\alpha_1]\varphi, [\alpha_1][\alpha_2]\varphi \Rightarrow [\alpha_1]\psi_2}{[\alpha_1]\varphi \wedge [\alpha_1][\alpha_2]\varphi \Rightarrow [\alpha_1]\psi_2} (w) \\
\frac{[\alpha_1]\varphi \wedge [\alpha_1][\alpha_2]\varphi \Rightarrow \psi_1}{[\alpha_1; \alpha_2]\varphi \Rightarrow \psi_1} ([\cdot]; [\cdot] \uparrow) \quad \frac{[\alpha_1]\varphi \wedge [\alpha_1][\alpha_2]\varphi \Rightarrow [\alpha_1]\psi_2}{[\alpha_1; \alpha_2]\varphi \Rightarrow [\alpha_1]\psi_2} (\wedge l) \\
\frac{[\alpha_1; \alpha_2]\varphi \Rightarrow \psi_1}{[\alpha_1; \alpha_2]\varphi \Rightarrow \psi_1 \wedge [\alpha_1]\psi_2} (\wedge r)
\end{array}$$

and

$$\begin{array}{c}
\frac{\psi_1 \Rightarrow [\alpha_1]\varphi}{\psi_1, [\alpha_1]\psi_2 \Rightarrow [\alpha_1]\varphi} (w) \quad \frac{\psi_2 \Rightarrow [\alpha_2]\varphi}{[\alpha_1]\psi_2 \Rightarrow [\alpha_1][\alpha_2]\varphi} (gen) \\
\frac{\psi_1, [\alpha_1]\psi_2 \Rightarrow [\alpha_1]\varphi}{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1]\varphi} (\wedge l) \quad \frac{\psi_1, [\alpha_1]\psi_2 \Rightarrow [\alpha_1][\alpha_2]\varphi}{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1][\alpha_2]\varphi} (w) \\
\frac{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1]\varphi}{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1]\varphi \wedge [\alpha_1][\alpha_2]\varphi} (\wedge l) \quad \frac{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1][\alpha_2]\varphi}{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1][\alpha_2]\varphi} (\wedge r) \\
\frac{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1]\varphi \wedge [\alpha_1][\alpha_2]\varphi}{\psi_1 \wedge [\alpha_1]\psi_2 \Rightarrow [\alpha_1; \alpha_2]\varphi} ([\cdot]; [\cdot] \downarrow)
\end{array}$$

Hence $\vdash [\alpha_1; \alpha_2]\varphi \Leftrightarrow \psi_1 \wedge [\alpha_1]\psi_2$. Since ψ_1 and ψ_2 are PDL formulas, so is $\psi_1 \wedge [\alpha_1]\psi_2$.

The case for $\alpha = \alpha_1 \cup \alpha_2$ is similar to $\alpha = \alpha_1; \alpha_2$ and we omit it here.

If $\alpha = \alpha_1^*$, by induction on the structure of regular programs, there is a PDL formula ψ such that $[\alpha_1]\varphi \vdash \psi$. From the sequent $[\alpha_1]\varphi \Rightarrow \psi$, we can make the following derivations:

$$\begin{array}{c}
\frac{[\alpha_1]\varphi \Rightarrow \psi}{[\alpha_1^*][\alpha_1]\varphi \Rightarrow [\alpha_1^*]\psi} ([gen]) \quad \frac{\psi \Rightarrow [\alpha_1]\varphi}{[\alpha_1^*]\psi \Rightarrow [\alpha_1^*][\alpha_1]\varphi} ([gen]) \\
\frac{[\alpha_1^*][\alpha_1]\varphi \Rightarrow [\alpha_1^*]\psi}{[\alpha_1^*]\varphi \Rightarrow [\alpha_1^*]\psi} ([\cdot]^* \uparrow) \quad \text{and} \quad \frac{[\alpha_1^*]\psi \Rightarrow [\alpha_1^*][\alpha_1]\varphi}{[\alpha_1^*]\psi \Rightarrow [\alpha_1^*]\varphi} ([\cdot]^* \downarrow)
\end{array}$$

So $\vdash [\alpha_1^*]\varphi \Leftrightarrow [\alpha_1^*]\psi$. Since ψ is a PDL formula, so is $[\alpha_1^*]\psi$. $\square$

Appendix C. Proofs of other propositions

Proof of Proposition 1. We proceed by induction on the syntactic structure of program α , where the base case is obvious. Below we only consider the case when $\alpha = \alpha_1^*$, other cases when $\alpha = (\alpha_1; \alpha_2)$ and $\alpha = (\alpha_1 \cup \alpha_2)$ are similar.

For any $(s, t) \in m(\alpha_1^*)$, by the definition of m there exists $n \geq 0$ such that $(s, t) \in m(\alpha_1^n)$. If $n = 0$ or $n = 1$, the result is direct by inductive hypothesis. If $n > 1$, there exist $u_1, \dots, u_{n-1}$ such that $(s, u_1) \in m(\alpha)$, $(u_i, u_{i+1}) \in m(\alpha)$ for all $1 \leq i \leq n-2$, and $(u_{n-1}, t) \in m(\alpha)$. By inductive hypothesis and the definition of Re , it is not hard to see that $(s, t) \in Re(\alpha^n)$. So $(s, t) \in Re(\alpha^*)$. $\square$

Proof of Proposition 2. For a PDL_b formula ϕ , we proceed by induction on the number of appearances of the modality $[\cdot]$ in ϕ . In the base case, there are no appearances of $[\cdot]$ in ϕ , so ϕ is already a PDL formula. In the inductive case, we only analyze the case when $\phi = [\alpha]\psi$, other cases are trivial.

In the following, we use $\varphi^\sharp$ to emphasize that formula φ is a PDL formula.

Since the number of appearances of $[\cdot]$ in ψ is strictly less than that of $[\alpha]\psi$, by inductive hypothesis, ψ is expressible. So there exists a PDL formula $\varphi^\sharp$ such that $\models \psi \leftrightarrow \varphi^\sharp$. Therefore $\models [\alpha]\psi \leftrightarrow [\alpha]\varphi^\sharp$ holds, which means that if $[\alpha]\varphi^\sharp$ is expressible, so is $[\alpha]\psi$. Next we prove that $[\alpha]\varphi^\sharp$ is expressible by induction on the structure of α .

Below by Theorem 1, from the soundness of each rule we can directly use the logical equivalence between the premise and conclusion of each rule.

If $\alpha = a$, by the semantics of a and rule $([a])$ we have $\models [a]\varphi^\sharp \leftrightarrow \varphi^\sharp \wedge [a]\varphi^\sharp$. Obviously, $\varphi^\sharp \wedge [a]\varphi^\sharp$ is a FODL formula. The cases when $\alpha = \alpha_1; \alpha_2$, $\alpha = \alpha_1 \cup \alpha_2$ and $\alpha = \alpha_1^*$ are similar, we only give the proof of the case when $\alpha = \alpha_1; \alpha_2$. From the semantics of $\alpha_1; \alpha_2$ and rule $([\cdot]; [\cdot])$, actually we have $\models [\alpha_1; \alpha_2]\varphi^\sharp \leftrightarrow [\alpha_1]\varphi^\sharp \wedge [\alpha_1][\alpha_2]\varphi^\sharp$. Since α_1 and α_2 have simpler structures than $\alpha_1; \alpha_2$, by inductive hypothesis, we know that $[\alpha_1]\varphi^\sharp$ and $[\alpha_2]\varphi^\sharp$ are expressible in PDL, so is $[\alpha_1]\varphi^\sharp \wedge [\alpha_1][\alpha_2]\varphi^\sharp$. $\square$

Lemma 2 ("Non-aborting" property). Given a while program α , for any state $s \in \mathbf{S}$, there exists a trace $tr \in m_{tr}(\alpha)$ such that $tr^b = s$.

Proof. We proceed by induction on the structure of α , where the base case (when $\alpha = (x := e)$) is trivial. Below we only consider the inductive cases.

When $\alpha = (\alpha_1; \alpha_2)$, by inductive hypothesis, there exists a trace $tr_1 \in m_{tr}(\alpha_1)$ such that $tr_1^b = s$, and there exists a trace $tr_2 \in m_{tr}(\alpha_2)$ such that $tr_2^b = tr_1^e$. So there exists a trace: $tr = tr_1 \circ tr_2$ such that $tr^b = s$, and it is obvious that $tr \in m_{tr}(\alpha_1; \alpha_2)$.

When $\alpha = (\text{if } \psi \text{ then } \alpha_1 \text{ else } \alpha_2)$, by inductive hypothesis there exist traces $tr_1 \in m_{tr}(\alpha_1)$ and $tr_2 \in m_{tr}(\alpha_2)$ such that $tr_1^b = tr_2^b = s$ holds. If $s \models \psi$, then $tr_1 \in m_{tr}(\text{if } \psi \text{ then } \alpha_1 \text{ else } \alpha_2)$; otherwise, $tr_2 \in m_{tr}(\text{if } \psi \text{ then } \alpha_1 \text{ else } \alpha_2)$.

When $\alpha = (\text{while } \psi \text{ do } \alpha_1)$, starting from s , we can construct a sequence of traces $tr_1, tr_2, \dots, tr_i \in m_{tr}(\alpha_1)$ ($i \geq 1$), where $tr_1^b = s$, $tr_i^e = tr_{i+1}^b$ and $tr_i^b \models \psi$, by repeatedly applying inductive hypothesis. In this construction procedure, if there is some $k \geq 0$ such that $tr_k^e \models \neg\psi$, then we get a trace $tr = tr_1 \circ \dots \circ tr_k \in m_{tr}(\text{while } \psi \text{ do } \alpha_1)$; if for all $k \geq 0$, $tr_k^e \models \psi$, then we get an infinite trace $tr = tr_1 \circ \dots \circ tr_k \circ \dots \in (\text{while } \psi \text{ do } \alpha_1)$; if there is some $k \geq 0$ such that tr_k is infinite, then we get a (infinite) trace $tr = tr_1 \circ \dots \circ tr_k \in m_{tr}(\text{while } \psi \text{ do } \alpha_1)$. $\square$

Lemma 3. Given a DLT program α , there exists a regular program β such that

- (i) for any trace $tr \in m_{tr}(\alpha)$, $(tr^b, tr^i) \in Re(\beta)$ for all $i \geq 0$ and if tr is finite, $(tr^b, tr^e) \in m(\beta)$;
- (ii) for any pair $(s, t) \in Re(\beta)$, there exists a trace tr such that $tr \in m_{tr}(\alpha)$, $tr^b = s$, and $tr^i = t$ for some $i \geq 0$. Moreover, if $(s, t) \in m(\beta)$, we can find a finite such tr and i is the last element of tr , in other words, $tr^e = t$.

Proof. We prove by induction on the structure of α .

- For the base case $\alpha = (x := e)$, let $\beta = (x := e)$. On one hand, for any trace $ss' \in m_{tr}(x := e)$, by the definition of $Re(x := e)$ given in Definition 7, we see that $(s, s), (s, s') \in Re(x := e)$ and $(s, s') \in m(x := e)$. On the other hand, for any $(u, v) \in Re(x := e)$, either $(u, v) \in m(tt?)$ (in this case $v = u$) or $(u, v) \in m(x := e)$ holds. If $(u, v) \in m(tt?)$, then $u = v$. Let $u' = u[x \mapsto I_u(e)]$, then according to the definition of m_{tr} in Sect. 6.1, the trace uu' satisfies $uu' \in m_{tr}(x := e)$ and the condition (ii) of Sect. 6.1. If $(u, v) \in m(x := e)$, then the trace uv itself is the one required as it is sure that $uv \in m_{tr}(x := e)$.
- For sequence program $\alpha = (\alpha_1 ; \alpha_2)$, by inductive hypothesis, there exist β_1 and β_2 corresponding to α_1 and α_2 respectively which satisfy conditions (i) and (ii). Let $\beta = (\beta_1 ; \beta_2)$, now we prove that α and β satisfy the conditions (i) and (ii).

On one hand, for any trace $tr \in m_{tr}(\alpha_1 ; \alpha_2)$, let $tr = tr_1 \circ tr_2$ with $tr_1 \in m_{tr}(\alpha_1)$ and $tr_2 \in m_{tr}(\alpha_2)$. If tr_1 is infinite, then $tr_1 \circ tr_2 = tr_1$. By condition (i), we directly have that $(tr^b, tr^i) \in Re(\beta_1) \subseteq Re(\beta_1 ; \beta_2)$ for all $i \geq 0$. If tr_1 is finite, by condition (i), we have that $(tr_1^b, tr_1^i) \in Re(\beta_1)$ for all $i \geq 0$, $(tr_2^b, tr_2^k) \in Re(\beta_2)$ for all $k \geq 0$ and $(tr_1^b, tr_1^e) \in m(\beta_1)$. Therefore, by the definition of $Re(\beta_1 ; \beta_2)$ (Definition 7), we know that $(tr^b, tr^j) \in Re(\beta_1 ; \beta_2)$ for all $j \geq 0$.

On the other hand, for any pair $(s, t) \in Re(\beta_1 ; \beta_2)$, either $(s, t) \in Re(\beta_1)$ or $(s, t) \in m(\beta_1) \circ Re(\beta_2)$ holds.

- If $(s, t) \in Re(\beta_1)$, by condition (ii), there exists a trace $tr \in \alpha_1$ such that $tr^b = s$ and $tr^i = t$ for some $i \geq 0$. If tr is infinite, then $tr \in m_{tr}(\alpha_1 ; \alpha_2)$ and tr is the trace we look for. If tr is finite, by Lemma 3, we see that there must exist a trace $tr' \in m_{tr}(\alpha_1 ; \alpha_2)$ whose prefix is tr , because from tr^e we can always find a trace in α_2 . Hence, $tr'^b = s$ and $tr'^j = t$ for some $j \geq 0$.
- If $(s, t) \in m(\beta_1) \circ Re(\beta_2)$, then there exists a state u such that $(s, u) \in m(\beta_1)$ and $(u, t) \in Re(\beta_2)$. By condition (ii), there exist trace tr_1 and trace tr_2 such that $tr_1^b = s$, $tr_1^e = u$, $tr_2^b = u$ and $tr_2^k = t$ for some $k \geq 0$. Let $tr = tr_1 \circ tr_2$, we have obtained the result.

If the pair $(s, t) \in m(\beta_1 ; \beta_2)$, there exists a state u such that $(s, u) \in m(\beta_1)$ and $(u, t) \in m(\beta_2)$. By condition (ii) there exist trace tr_1 and tr_2 such that $tr_1^b = s$, $tr_1^e = u$, $tr_2^b = u$ and $tr_2^e = t$. Then $tr = tr_1 \circ tr_2$ is the trace we require.

- For if...then...else... statement $\alpha = (\text{if } \psi \text{ then } \alpha_1 \text{ else } \alpha_2)$, by inductive hypothesis, there exist β_1 and β_2 corresponding to α_1 and α_2 respectively which satisfy conditions (i) and (ii). Let $\beta = (\psi? ; \beta_1 \cup \neg\psi? ; \beta_2)$, we prove that α and β satisfy conditions (i) and (ii).

On one hand, for any trace $tr \in m_{tr}(\text{if } \psi \text{ then } \alpha_1 \text{ else } \alpha_2)$, either $tr \in m_{tr}(\alpha_1)$ (if $tr^b \models \psi$), or $tr \in m_{tr}(\alpha_2)$ (if $tr^b \models \neg\psi$) holds. If $tr \in m_{tr}(\alpha_1)$, by condition (i) $(tr^b, tr^i) \in Re(\beta_1)$ for all $i \geq 0$. Since $tr^b \models \psi$, $(tr^b, tr^i) \in Re(\psi? ; \beta_1)$ also holds, which means $(tr^b, tr^i) \in Re(\psi? ; \beta_1 \cup \neg\psi? ; \beta_2)$ for all $i \geq 0$. If tr is finite, again by condition (i) $(tr^b, tr^e) \in m(\beta_1)$. Then we also have $(tr^b, tr^e) \in m(\psi? ; \beta_1) \subseteq m(\psi? ; \beta_1 \cup \neg\psi? ; \beta_2)$. The case for $tr \in m_{tr}(\alpha_2)$ is similar and we omit it here.

On the other hand, for any pair $(s, t) \in Re(\psi? ; \beta_1 \cup \neg\psi? ; \beta_2)$, either $(s, t) \in Re(\psi? ; \beta_1)$ or $(s, t) \in Re(\neg\psi? ; \beta_2)$. If $(s, t) \in Re(\psi? ; \beta_1)$, which is equivalent to $(s, t) \in Re(\beta_1)$ and $s \models \psi$, by condition (ii), we know that there exists a trace $tr \in m_{tr}(\alpha_1)$ such that $tr^b = s$ and $tr^i = t$ for some $i \geq 0$. Since $s \models \psi$, $tr \in m_{tr}(\text{if } \psi \text{ then } \alpha_1 \text{ else } \alpha_2)$. The case for $(s, t) \in Re(\neg\psi? ; \beta_2)$ is similar. If the pair $(s, t) \in m(\psi? ; \beta_1 \cup \neg\psi? ; \beta_2)$, either $(s, t) \in m(\psi? ; \beta_1)$ or $(s, t) \in m(\neg\psi? ; \beta_2)$ holds. We take $(s, t) \in m(\psi? ; \beta_1)$ for example, the other case is similar. We have that $(s, t) \in m(\beta_1)$ and $s \models \psi$ hold. Again by condition (ii) there is a trace $tr \in m_{tr}(\alpha_1)$ that satisfies $tr^b = s$ and $tr^e = t$. Since $s \models \psi$, also $tr \in m_{tr}(\text{if } \psi \text{ then } \alpha_1 \text{ else } \alpha_2)$ holds.

- For while statement $\alpha = (\text{while } \psi \text{ do } \alpha_1)$, by inductive hypothesis, there exists a β_1 corresponding to α_1 which satisfies conditions (i) and (ii). Let $\beta = ((\psi? ; \beta_1)^* ; \neg\psi?)$, we prove that α and β satisfy conditions (i) and (ii).

On one hand, for any trace $tr \in m_{tr}(\text{while } \psi \text{ do } \alpha_1)$, according to the definition of m_{tr} in Sect. 6.1, there are 3 cases as below.

- In the case (i) of Sect. 6.1, we consider two subcases:
 - * If $tr = ss$ where $s \models \neg\psi$, trivially we have $(s, s) \in m(\neg\psi?)$ so $(s, s) \in m((\psi? ; \beta_1)^* ; \neg\psi?)$.
 - * If $tr = tr_1 \circ \dots \circ tr_n$, where $tr_i \in m_{tr}(\alpha_1)$ is finite, $tr_i^b \models \psi$ for all $i \geq 0$, and $tr_n^e \models \neg\psi$, by condition (i), $(tr_i^b, tr_i^k) \in Re(\beta_1)$ for all $i, k \geq 0$ and $(tr_i^b, tr_i^e) \in m(\beta_1)$ for all $i \geq 0$ since tr_i is finite. By Definition 7, it is not hard to see that $(tr^b, tr^k) \in Re((\psi? ; \beta_1)^*)$ for any i, k (not including tr_n^e), and $(tr^b, tr_n^e) \in m((\psi? ; \beta_1)^* \circ Re(\neg\psi?))$. Therefore

- $(tr^b, tr^k) \in Re((\psi?; \beta_1)^*; \neg\psi?)$ for all $k \geq 0$. From $(tr_i^b, tr_i^e) \in m(\beta_1)$ for any i and the fact that $tr_n^e \models \neg\psi$, we can get $(tr^b, tr^e) \in m((\psi?; \beta_1)^*; \neg\psi?)$.
- If $tr = tr_1 \circ \dots \circ tr_n \circ \dots$, where $tr_i \in m_{tr}(\alpha_1)$ is finite and $tr_i^b \models \psi$ for all $i \geq 1$ (the case (ii) of Sect. 6.1), similar as above, by condition (i) we can obtain that $(tr_i^b, tr_i^k) \in Re(\beta_1)$ and $(tr_i^b, tr_i^e) \in m(\beta_1)$ for any $i, k \geq 0$. Similarly, by Definition 7, it is not hard to get that $(tr^b, tr^k) \in Re((\psi?; \beta_1)^*)$ for any i, k . Therefore $(tr^b, tr^k) \in Re((\psi?; \beta_1)^*; \neg\psi?)$ for all $k \geq 0$.
 - If $tr = tr_1 \circ \dots \circ tr_n$ where $tr_i \in m_{tr}(\alpha_1)$ is finite for all $1 \leq i < n$ and $tr_n \in m_{tr}(\alpha_1)$ is infinite, and $tr_i \models \psi$ for all $i \geq 1$ (the case (iii) of Sect. 6.1), by condition (i), we know that $(tr_i^b, tr_i^k) \in Re(\beta_1)$ for any $i, k \geq 0$, and $(tr_i^b, tr_i^e) \in m(\beta_1)$ for any $i < n$. Like above, by Definition 7, we can get that $(tr^b, tr_i^k) \in Re((\psi?; \beta_1)^*)$ for any $i, k \geq 0$, which means $(tr^b, tr^k) \in Re((\psi?; \beta_1)^*; \neg\psi?)$ for all $k \geq 0$.
- On the other hand, for any pair $(s, t) \in Re((\psi?; \beta_1)^*; \neg\psi?)$, by Definition 7, either $(s, t) \in Re((\psi?; \beta_1)^*)$ or $(s, t) \in m((\psi?; \beta_1)^*) \circ Re(\neg\psi?)$ holds.
- If $(s, t) \in Re((\psi?; \beta_1)^*)$, by Definition 7, it is not hard to see that either $s = t$ and $(s, s) \in Re(tt?)$ or there exist $u_1, \dots, u_n$ ($n \geq 1$) with $u_1 = s$ such that $(u_i, u_{i+1}) \in m(\psi?; \beta_1)$ for all $1 \leq i < n$ and $(u_n, t) \in Re(\psi?; \beta_1)$. The case for $s = t$ is trivial, in which we let $tr = ss$. From $(u_i, u_{i+1}) \in m(\psi?; \beta_1)$ and $(u_n, t) \in Re(\psi?; \beta_1)$, we can get that $(u_i, u_{i+1}) \in m(\beta_1)$ and $u_i \models \psi$ for all $i < n$, $(u_n, t) \in Re(\beta_1)$ and $u_n \models \psi$. By condition (ii), there exist finite traces $tr_1, \dots, tr_{n-1} \in m_{tr}(\alpha_1)$ such that $tr_i^b = u_i$ and $tr_i^e = u_{i+1}$ for all $0 \leq i < n$, and a trace $tr_n \in m_{tr}(\alpha_1)$ such that $tr_n^b = u_n$ and $tr_n^k = t$ for some $k \geq 0$. Let $tr = tr_1 \circ \dots \circ tr_n$, if tr_n is infinite or finite with $tr_n^e \models \neg\psi$, then $tr \in m_{tr}(\text{while } \psi \text{ do } \alpha_1)$. If tr_n is finite with $tr_n^e \models \psi$, by Lemma 3, there exists a trace $tr' \in m_{tr}(\text{while } \psi \text{ do } \alpha_1)$ whose prefix is tr , because from tr_n^e there must exist a trace in $\text{while } \psi \text{ do } \alpha_1$. Since t is one of states of tr , it is also one of states of tr' .
 - The case for $(s, t) \in m((\psi?; \beta_1)^*) \circ Re(\neg\psi?)$ can be proved in a similar way. $\square$

Before proving Proposition 7, we need a result about the expressibility of FODL in AFOL (cf. [29]) stated as follows.

Proposition 10 (Expressibility of FODL in AFOL [29]). *For any FODL formula ϕ , there is a AFOL formula ψ such that $\models \phi \leftrightarrow \psi$.*

Proof of Proposition 7. We firstly prove that for any DLT formula $\llbracket \alpha \rrbracket \phi^b$, there exists a regular program β such that

$$\models \llbracket \alpha \rrbracket \phi^b \leftrightarrow \llbracket \beta \rrbracket \phi^b, \quad (\text{C.1})$$

where we use the superfix b to emphasize that a formula is a AFOL formula.

For any program α , according to Lemma 3, there exists a regular program β such that the conditions (i) and (ii) of Lemma 3 hold. Now we prove for any state s , $s \models \llbracket \alpha \rrbracket \phi^b$ iff $s \models \llbracket \beta \rrbracket \phi^b$. On one hand, we assume $s \models \llbracket \alpha \rrbracket \phi^b$. For any pair $(s, t) \in Re(\beta)$, by Lemma 3 there exists a trace $tr \in m_{tr}(\alpha)$ such that $tr^b = s$ and $tr^i = t$ for some $i \geq 0$. So $t \models \phi^b$ because $s \models \llbracket \alpha \rrbracket \phi^b$. Hence $s \models \llbracket \beta \rrbracket \phi^b$. On the other hand, suppose $s \models \llbracket \beta \rrbracket \phi^b$. For any trace $tr \in m_{tr}(\alpha)$, by Lemma 3 we have $(tr^b, tr^i) \in Re(\beta)$ for any $i \geq 0$. Since $s \models \llbracket \beta \rrbracket \phi^b$, $tr^i \models \phi^b$ for any $i \geq 0$. Therefore $s \models \llbracket \alpha \rrbracket \phi^b$.

Now we prove: Given a DLT formula ψ , there exists a FODL_b formula φ such that $\models \psi \leftrightarrow \varphi$.

We proceed by induction on the number n of appearances of modality $\llbracket \cdot \rrbracket$ in a DLT formula ψ . When $n = 0$, ψ is a AFOL formula. Since DLT and FODL_b share the same set of AFOL formulas, the proof is trivial as the FODL_b formula required is exactly ψ itself.

When $n > 0$, the only non-trivial case is when ψ is a dynamic formula of the form $\llbracket \alpha \rrbracket \psi'$. By inductive hypothesis, there exists a FODL_b formula φ' such that $\models \psi' \leftrightarrow \varphi'$. By Proposition 5 and Proposition 10, we know that FODL_b is also expressible in AFOL, so there exists a AFOL formula φ_1^b such that $\models \varphi' \leftrightarrow \varphi_1^b$. Hence, we have $\models \psi' \leftrightarrow \varphi_1^b$, therefore $\models \llbracket \alpha \rrbracket \psi' \leftrightarrow \llbracket \alpha \rrbracket \varphi_1^b$. By the result (C.1) above, there exists a regular program β such that $\models \llbracket \alpha \rrbracket \varphi_1^b \leftrightarrow \llbracket \beta \rrbracket \varphi_1^b$. So $\models \llbracket \alpha \rrbracket \psi' \leftrightarrow \llbracket \beta \rrbracket \varphi_1^b$. On the other hand, from $\models \varphi' \leftrightarrow \varphi_1^b$, there is $\models \llbracket \beta \rrbracket \varphi' \leftrightarrow \llbracket \beta \rrbracket \varphi_1^b$ by the soundness of rule (gen). Therefore, we obtain the result $\models \llbracket \alpha \rrbracket \psi' \leftrightarrow \llbracket \beta \rrbracket \varphi'$, where clearly $\llbracket \beta \rrbracket \varphi'$ is a FODL_b formula. $\square$

References

- [1] K.R. Apt, F.S. de Boer, E.-R. Olderog, *Verification of Sequential and Concurrent Programs*, Texts in Computer Science, Springer, 2009.
- [2] C. Baier, J.P. Katoen, *Principles of Model Checking*, The MIT Press, 2008.
- [3] C.A.R. Hoare, An axiomatic basis for computer programming, *Commun. ACM* 12 (1969) 576–580.
- [4] B. Beckert, V. Klebanov, B. Weiß, *Dynamic Logic for Java*, Springer International Publishing, Cham, 2016, pp. 49–106.
- [5] M.R.F. Benevides, L.M. Schechter, A propositional dynamic logic for CCS programs, in: W. Hodges, R. de Queiroz (Eds.), *Logic, Language, Information and Computation*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2008, pp. 83–97.
- [6] M.R. Benevides, L.M. Schechter, A propositional dynamic logic for concurrent programs based on the π -calculus, in: *Proceedings of the 6th Workshop on Methods for Modalities, M4M-6 2009*, in: *Electronic Notes in Theoretical Computer Science*, vol. 262, 2010, pp. 49–64.
- [7] A. Platzer, Differential dynamic logic for verifying parametric hybrid systems, in: *International Conference on Theorem Proving with Analytic Tableaux and Related Methods (TABLEAUX)*, in: *Lecture Notes in Computer Science (LNCS)*, vol. 4548, Springer, Berlin Heidelberg, 2007, pp. 216–232.
- [8] A. Platzer, *Logical Foundations of Cyber-Physical Systems*, Springer, Cham, 2018.
- [9] Y.A. Feldman, D. Harel, A probabilistic dynamic logic, *J. Comput. Syst. Sci.* 28 (1984) 193–215.
- [10] A. Platzer, Stochastic differential dynamic logic for stochastic hybrid programs, in: N. Björner, V. Sofronie-Stokkermans (Eds.), *Automated Deduction – CADE-23*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 446–460.

- [11] H. Ditmarsch, W. Hoek, B. Kooi, *Dynamic Epistemic Logic*, Springer, 2008.
- [12] P. Marc, P. Rohit, Game logic - an overview, *Stud. Log.* 75 (2003) 165–182.
- [13] B. Bohrer, A. Platzer, Constructive hybrid games, in: N. Peltier, V. Sofronie-Stokkermans (Eds.), *Automated Reasoning*, Springer International Publishing, Cham, 2020, pp. 454–473.
- [14] A. Pnueli, The temporal logic of programs, in: 18th Annual Symposium on Foundations of Computer Science (SFCS 1977), 1977, pp. 46–57.
- [15] D. Harel, D. Kozen, R. Parikh, Process logic: expressiveness, decidability, completeness, *J. Comput. Syst. Sci.* 25 (1982) 144–170.
- [16] M.J. Fischer, R.E. Ladner, Propositional dynamic logic of regular programs, *J. Comput. Syst. Sci.* 18 (1979) 194–211.
- [17] B. Beckert, S. Schlager, A sequent calculus for first-order dynamic logic with trace modalities, in: R. Goré, A. Leitsch, T. Nipkow (Eds.), *Automated Reasoning*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2001, pp. 626–641.
- [18] M. Heisel, W. Reif, W. Stephan, A dynamic logic for program verification, in: A.R. Meyer, M.A. Taitlin (Eds.), *Logic at Botik '89*, Springer Berlin Heidelberg, Berlin, Heidelberg, 1989, pp. 134–145.
- [19] K. Rustan, M. Leino, *Verification of Object-Oriented Software. The KeY Approach*, Lecture Notes in Computer Science (LNCS), vol. 4334, Springer, 2007.
- [20] R. Hennicker, M. Wirsing, A dynamic logic for systems with predicate-based communication, in: T. Margaria, B. Steffen (Eds.), *Leveraging Applications of Formal Methods, Verification and Validation: Engineering Principles*, Springer International Publishing, Cham, 2020, pp. 224–242.
- [21] Y. Zhang, F. Mallet, H. Zhu, Y. Chen, B. Liu, Z. Liu, A clock-based dynamic logic for schedulability analysis of ccs1 specifications, *Sci. Comput. Program.* 202 (2021) 102546, <https://doi.org/10.1016/j.scico.2020.102546>.
- [22] M.H. ter Beek, G. Cledou, R. Hennicker, J. Proença, Can we communicate? Using dynamic logic to verify team automata, in: M. Chechik, J.-P. Katoen, M. Leucker (Eds.), *Formal Methods*, Springer International Publishing, Cham, 2023, pp. 122–141.
- [23] N. Halbwachs, *Synchronous Programming of Reactive Systems*, Kluwer Academic Pub., 1993.
- [24] R. Parikh, A decidability result for a second order process logic, in: 19th Annual Symposium on Foundations of Computer Science, Sfcs 1978, 1978, pp. 177–183.
- [25] B. Beckert, D. Bruns, Dynamic logic with trace semantics, in: M.P. Bonacina (Ed.), *Automated Deduction – CADE-24*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2013, pp. 315–329.
- [26] A. Platzer, A temporal dynamic logic for verifying hybrid system invariants, in: S.N. Artemov, A. Nerode (Eds.), *Logical Foundations of Computer Science*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2007, pp. 457–471.
- [27] J.-B. Jeannin, A. Platzer, dtl2: differential temporal dynamic logic with nested temporalities for hybrid systems, in: S. Demri, D. Kapur, C. Weidenbach (Eds.), *Automated Reasoning*, Springer International Publishing, Cham, 2014, pp. 292–306.
- [28] D. Harel, D. Kozen, J. Tiuryn, *Dynamic Logic*, MIT Press, 2000.
- [29] D. Harel, *First-Order Dynamic Logic*, Lecture Notes in Computer Science (LNCS), vol. 68, Springer, 1979.
- [30] G. Gentzen, *Untersuchungen über das logische Schließen*, Ph.D. thesis, NA Göttingen, 1934.
- [31] K. Gödel, Über formal unentscheidbare sätze der principia mathematica und verwandter systeme, *Monatshefte Math. Phys.* 38 (1931) 173–198.
- [32] R. Milner, A complete inference system for a class of regular behaviours, *J. Comput. Syst. Sci.* 28 (1984) 439–466.
- [33] D. Sangiorgi, *Introduction to Bisimulation and Coinduction*, Cambridge University Press, USA, 2011.